

HTML/CSS

---

# HTML/CSS

02 CSS 기초

## 목차 02 CSS 기초

---

1. CSS 소개 및 기본 문법
2. 기본 선택자와 복합 선택자
3. 가상 선택자와 스타일 규칙
4. 텍스트 스타일링
5. 박스 모델



## 목차 02 CSS 기초

---

6. 배경과 그라디언트

7. Flex Layout

8. Grid Layout

9. Position

# 1. CSS 소개 및 기본 문법





## 웹 페이지의 스타일링, 레이아웃 등의 디자인을 지정하는 언어

- **디자인과 스타일링** : CSS를 사용해 웹 페이지의 레이아웃, 색상, 글꼴, 크기 등을 지정해 웹사이트를 디자인
- **일관성과 유지 보수성** : CSS를 사용하면 스타일을 별도의 파일에 정의해 여러 페이지에 일관된 스타일 적용 가능하고, 스타일 변경이 필요할 때 스타일 시트만 수정해 일관성 유지 가능
- **접근성** : CSS를 사용해 웹 페이지의 구조와 스타일을 분리해 웹 접근성 향상



## 환영합니다

이것은 간단한 문단입니다.

더 알아보기

저작권 © 2023

CSS가 적용되지 않은 웹 페이지

## 환영합니다

이것은 간단한 문단입니다.

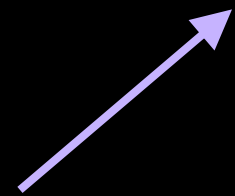
더 알아보기

저작권 © 2023

CSS가 적용된 웹 페이지



## Cascading Style Sheets



- '위에서 아래로 흐르는'이라는 의미
- 상위 요소의 스타일이 하위 요소에 상속되고,  
동일 요소에 여러 스타일 규칙이 있으면 우선순위에 따라 적용

**CSS는 웹 페이지의 다양한 요소에 스타일을 적용하는 규칙의 집합을 위에서 아래로 흐르듯이  
우선순위에 따라 순차적으로 적용하는 시스템을 의미**



```
선택자 {  
  속성 : 값 ;  
  속성 : 값 ;  
  /*  
  여러 개의  
  속성과 값들  
  */  
}
```

- **선택자(Selector)** : 스타일을 적용할 HTML 요소를 선택하는 부분
  - 원하는 요소를 선택하기 위해 태그 이름, 클래스, 아이디 등을 사용
  - 스타일을 적용할 대상을 지정하는 역할
- **선언(Declaration)** : 선택한 HTML 요소에 적용할 스타일을 정의하는 부분
  - "속성(Property) : 값(Value)" 의 형식으로 적용
  - 속성은 스타일을 어떻게 변경할지를 나타냄
  - 값은 해당 속성에 적용할 구체적인 값



```
/*  
선택자(태그 이름) 와  
선언(속성과 값)  
*/  
p {  
    color: blue;  
    font-size: 16px;  
}
```

<p>요소에 글자의 색상을 파랑색, 크기를 16px로 적용

- **선택자(Selector):** HTML 요소를 선택하는 부분
  - 예: **p** - 모든 **<p>** 요소 선택
- **속성(Property):** 스타일을 정의하는 부분
  - 예: **color** - 텍스트 색상 지정, **font-size** - 글꼴 크기 지정
- **값(Value):** 속성에 대한 설정 값
  - 예: **blue** - 파란색 텍스트, **16px** - 16px 크기 글꼴
- **세미콜론(;):** 각 속성-값 쌍 뒤에 사용
  - 다음 규칙의 시작을 나타냄
- **중괄호({}):** 규칙의 시작과 끝
  - 선택자와 규칙들을 감싸야 함

외부 스타일 시트

내부 스타일 시트

인라인 스타일

CSS를 HTML 문서에 적용하는 방법은 크게 3가지가 존재





```
<head>
  <link rel="stylesheet"
        href="style.css">
</head>
```

```
body {
  background-color: yellow;
}

p {
  color: red;
}
```

- CSS 코드를 별도의 파일로 저장하고, HTML 문서에서 외부 스타일 시트 파일을 참조하여 스타일을 적용
- 외부 스타일 시트 파일은 .css 확장자를 가지고, **<link>** 태그를 사용하여 HTML 문서와 연결
- 외부 스타일 시트를 사용하는 것이 **가장 권장되는 방법**
- 재사용성과 유지 보수성 측면에서 좋은 성능을 보여줌



- **<style>** 태그를 사용하여 HTML 문서 내부에 CSS 스타일을 정의 (<style>태그는 <head> 태그와 </head> 태그 내부에 위치)
- 이 방식 이용 시 해당 HTML 문서에서만 스타일 적용

```
<!DOCTYPE html>
<html lang="en">
<head>
  <style>
    p {
      color: blue;
      font-size: 16px;
    }
  </style>
</head>
<body>
  <p>
    이 문장은 내부 스타일
    시트로 적용됩니다.
  </p>
</body>
</html>
```

스타일 시트의 규칙이 적용되는 부분



```
<p style="color: red; font-size: 18px;">  
    이 문장은 인라인  
    스타일로 적용됩니다.  
</p>
```

- HTML 태그의 **style** 속성을 사용하여 해당 태그에 직접 스타일 지정
- 한 번 설정된 스타일을 변경하기가 매우 어렵고, 스타일 시트를 이용하는 많은 이점을 잃게 되기 때문에 **꼭 필요한 경우에만 사용**해야 함



- 스타일 적용 방법들이 혼합되어 사용될 경우, 최종적으로 적용되는 스타일은 우선순위는 다음과 같음

1. 인라인 스타일 (HTML 요소 내부에 위치)
2. 내부 / 외부 스타일 시트 (HTML 문서의 head 요소 내부에 위치)
3. 웹 브라우저 기본 스타일

- 인라인 스타일이 적용된 태그는 내부, 외부 스타일 시트에 상관없이 무조건 인라인 스타일 적용
- 내부 스타일 시트와 외부 스타일 시트는 가장 마지막에 적용된 스타일 시트가 적용



## 2. 기본 선택자와 복합 선택자



## CSS에서는 스타일을 적용할 대상을 선택하기 위해 다양한 선택자(selector) 사용

- 전체 선택자
- 태그 선택자
- 아이디(id) 선택자
- 클래스(class) 선택자



```
/* 전체 선택자를  
이용한 스타일 적용 */  
* {  
  margin: 0;  
  padding: 0;  
}
```

- 모든 HTML 요소에 스타일 적용 시 사용
- **\*(별표)**를 사용
- 초기화 스타일 시트(reset stylesheet) 만들거나 기본 스타일 설정 시 사용
- 모든 요소에 일괄적으로 스타일을 적용하는 것이 문제가 될 수 있으므로  
**사용에 주의**하여야 함.



```
/* 태그 선택자를  
이용한 스타일 적용 */  
h1 {  
  color: red;  
}  
  
p {  
  font-size: 16px;  
}
```

- HTML의 태그(p, h1, div 등)으로 요소를 선택해 스타일을 지정하는 선택자
- 특정 태그가 사용된 HTML 요소에 모두 동일한 스타일 적용





```
<body>
  <p>이것은 일반적인 문단입니다.</p>
  <p class="highlight">이 문단은 굵게 표시됩니다.</p>
  <p>이것도 일반적인 문단입니다.</p>
  <span class="highlight">이 스패 텍스트도 굵게 표시됩니다.</span>
</body>
```

```
/* 클래스 선택자를
이용한 스타일 적용 */
.highlight {
  font-weight: bold;
}
```

- `.`을 이용하여 클래스 이름으로 스타일 지정하는 선택자
- 동일한 클래스(같은 클래스 이름)를 가진 여러 요소에 같은 스타일 적용
- 같은 태그를 사용했어도 특정 요소에만 다른 스타일을 적용하고 싶을 때에도 사용



```
<body>
  <h1 id="title">
    제목입니다.
  </h1>
  <p>
    일반적인 문단입니다.
  </p>
</body>
```

```
/* ID 선택자를
이용한 스타일 적용 */
#title {
  font-size: 32px;
}
```

- `#`을 이용하여 ID 이름으로 스타일을 지정
- 하나의 요소에 대해 고유한 스타일 지정 시 사용
- 한 문서 내에서 **한 번만** 적용 가능



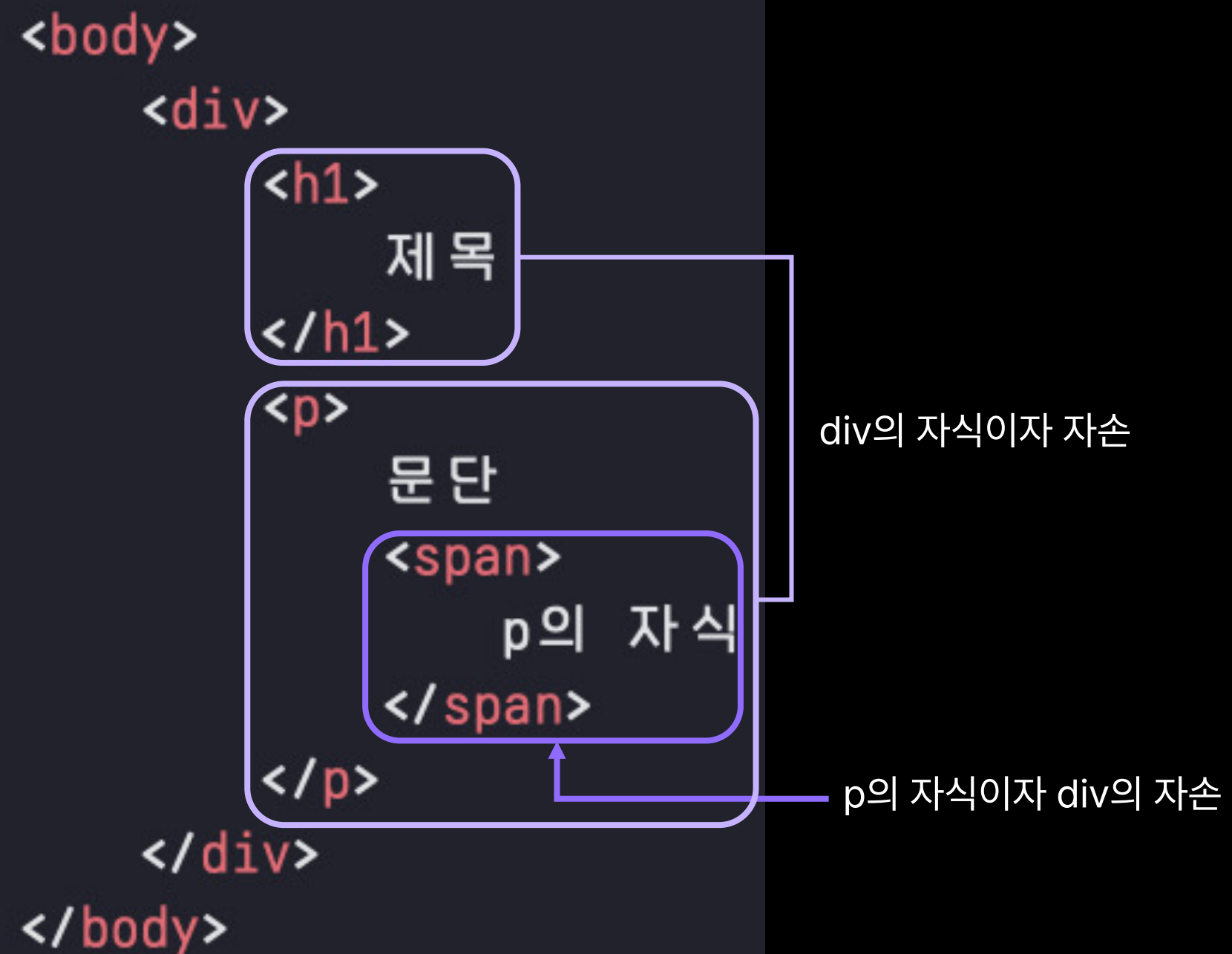
## 2개 이상의 단순 선택자를 조합해 특정 요소 선택 가능

- 자손 선택자
- 자식 선택자
- 그룹 선택자

자손 : 어떤 요소의 하위 요소(바로 아래 레벨의 요소 뿐만 아니라 모든 레벨의 하위 요소)

자식 : 어떤 요소의 바로 아래에 있는 하위요소(한 단계 아래 요소만 자식)

부모 : 특정 요소 바로 위에 있는 상위 요소



- h1은 div의 자식이자 자손
- p는 div의 자식, 자손이면서 span의 부모
- span은 p의 자식이자 div의 자손





```
<body>
  <div>
    <p>문 단1</p>
    <section>
      <p>문 단2</p>
    </section>
  </div>
  <p>문 단3</p>
</body>
```

```
/* 자손 선택자를
이용한 스타일 적용 */
div p {
  color: red;
}
```

- 공백을 사용하여 하위 요소를 선택
- 부모 요소와 하위 요소의 관계를 정확히 지정하여 스타일 적용
- 왼쪽 예시에서는 '문단1' 부분과 '문단2' 부분이 빨간색으로 바뀜



```
<body>
  <div>
    <p>파란색 문단</p>
    <section>
      <p>문단1</p>
    </section>
  </div>
  <p>문단2</p>
</body>
```

```
/* 자식 선택자를
이용한 스타일 적용 */
div > p {
  color: blue;
}
```

- `>`를 사용하여 직접적인 자식 요소를 선택
- **바로 아래의 자식 요소만** 스타일이 적용
- 왼쪽 예시의 경우 '파란색 문단' 부분만 파란색으로 표시됨



```
<body>
  <h1>제목 1</h1>
  <h2>제목 2</h2>
  <p>문단 1</p>
</body>
```

```
/* 그룹 선택자를
   이용한 스타일 적용 */
h1, h2, p {
  text-align: center;
  color: green;
}
```

- 심표(`,`)를 사용하여 여러 개의 선택자를 그룹으로 묶어 스타일 지정
- 같은 스타일을 여러 요소에 적용할 때 편리
- 왼쪽 예시의 경우 제목 1, 제목 2, 문단 1 모두 초록색으로 표시되고 가운데 정렬

### 3. 가상 선택자와 스타일 규칙



## 선택자에 추가하는 키워드로, 특정한 목적을 위해 사용되는 선택자

- 요소의 내부 구조나 상태에 따라 스타일을 다르게 적용하고자 할 때 유용
- HTML 문서의 수정 없이 CSS만으로 디자인적 요소를 추가할 수 있음

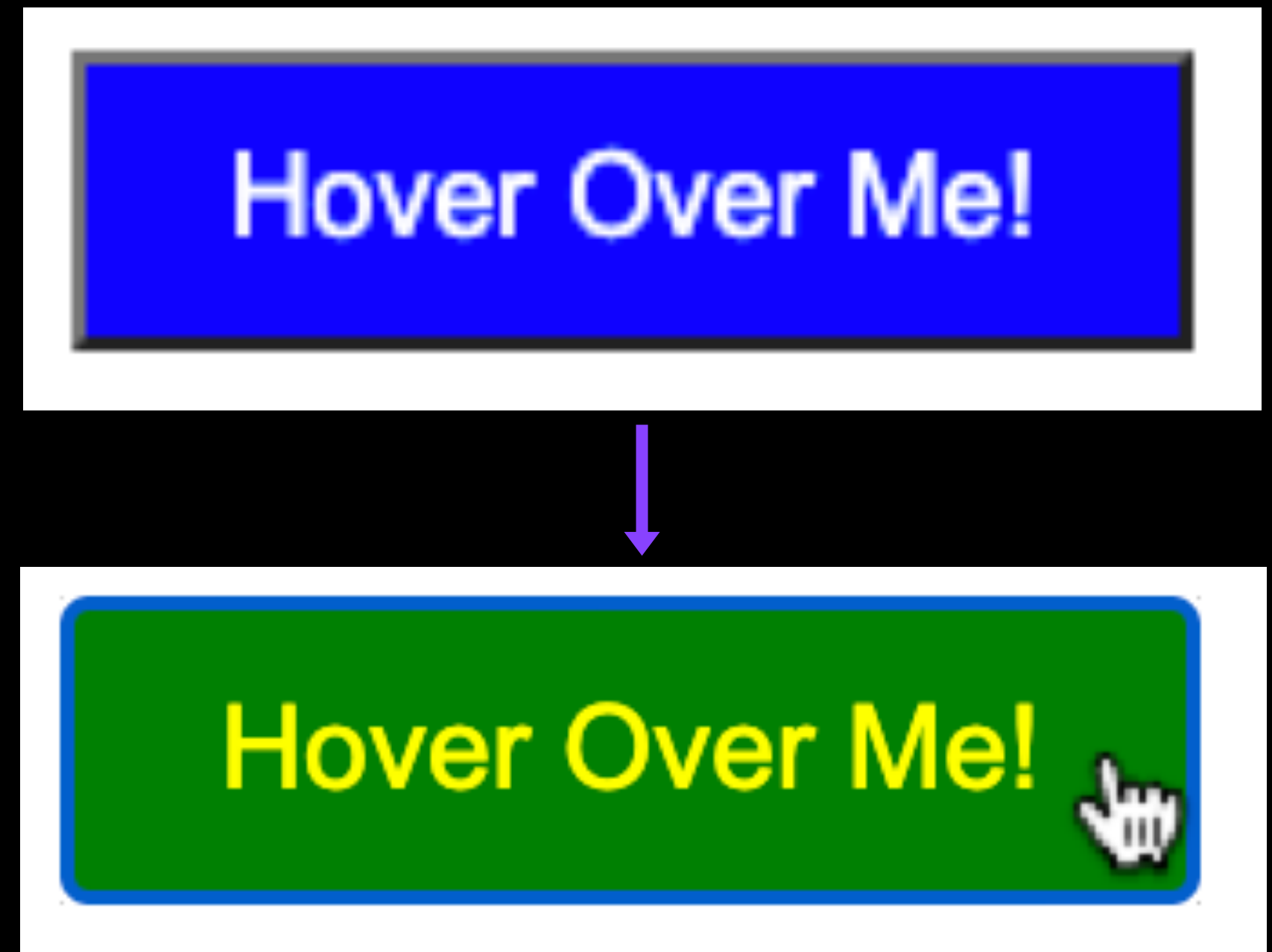
|            |                           |                                                                                                       |
|------------|---------------------------|-------------------------------------------------------------------------------------------------------|
| 가상 클래스 선택자 | 특정 상태에 있는 요소를 선택하여 스타일 적용 | <ul style="list-style-type: none"><li>• :hover</li><li>• :active</li><li>• :focus</li></ul>           |
| 가상 요소 선택자  | 특정 요소의 일부분에 스타일 적용        | <ul style="list-style-type: none"><li>• ::before</li><li>• ::after</li><li>• ::first-letter</li></ul> |



`:hover` : 마우스 커서가 요소 위에 있을 때 스타일이 적용

```
.hover-button {  
  background-color: blue;  
  color: white;  
  padding: 10px 20px;  
  font-size: 16px;  
  cursor: pointer;  
}  
  
.hover-button:hover {  
  background-color: green;  
  color: yellow;  
}
```

코드



적용 전 후



`:active` : 사용자가 요소를 활성화(클릭 등)했을 때 적용

```
.active-button {  
  background-color: red;  
  color: white;  
  padding: 10px 20px;  
  font-size: 16px;  
  cursor: pointer;  
}  
.active-button:active {  
  background-color: yellow;  
  color: black;  
}
```

코드



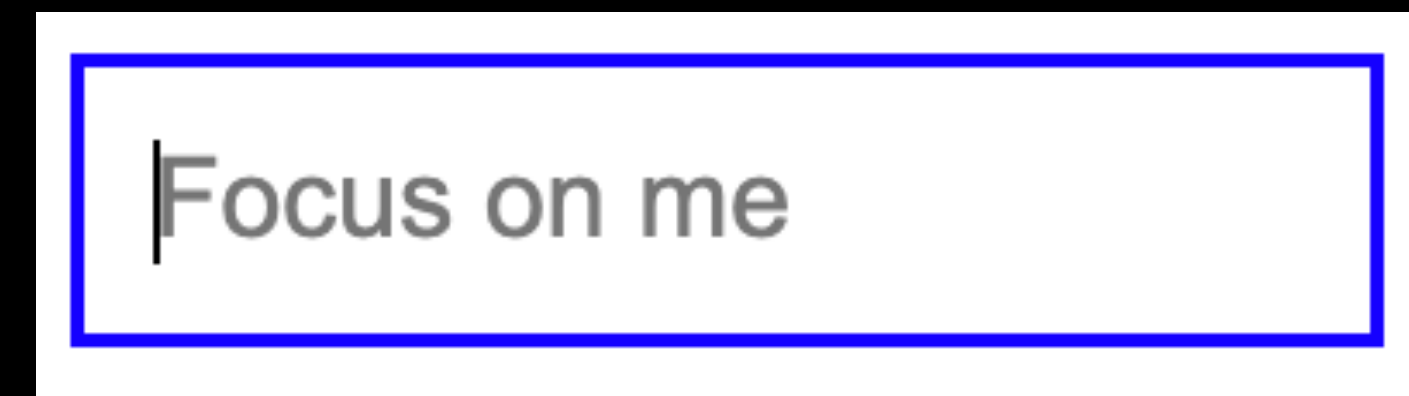
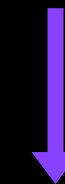
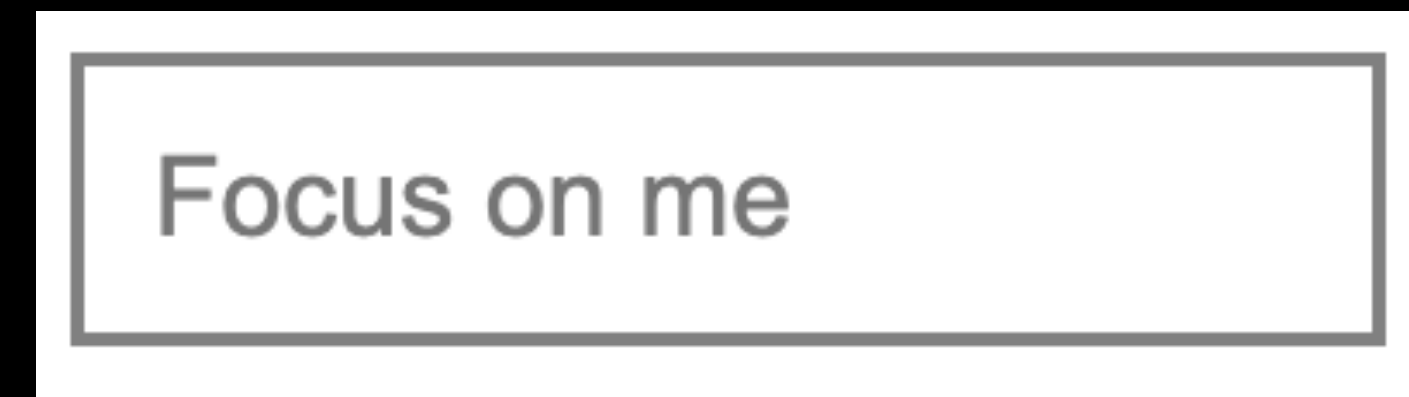
적용 전 후



`:focus` : 요소가 포커스를 받았을 때 적용(입력 필드에 커서가 들어간 경우 등)

```
.focus-input {  
  border: 2px solid grey;  
  padding: 10px;  
  font-size: 16px;  
}  
.focus-input:focus {  
  border: 2px solid blue;  
  outline: none;  
}
```

코드



적용 전 후





`::before` : 요소의 앞에 콘텐츠(지정 내용(텍스트나 이미지)) 추가

```
<style>
  .alert-box::before{
    content: "⚠ ";
    font-size: 1.5em;
  }
</style>
```

코드

주의사항입니다!



⚠ 주의사항입니다!

적용 전 후

`::first-letter` : 요소의 첫 글자에 적용

```
<style>
  .drop-cap::first-letter {
    font-size: 3em;
    font-weight: bold;
  }
</style>
```

코드

첫 번째 글자를 크게 만들어 보겠습니다.



**첫** 번째 글자를 크게 만들어 보겠습니다.

적용 전 후



우선순위가 높은 스타일이 먼저 적용되고,  
우선순위가 같을 경우 선택자의 개수가 많은 CSS가 우선 적용되고,  
이것도 동일할 경우 가장 뒤쪽에 작성된 스타일이 적용

- **!important > 인라인 선언 방식 > id 선택자 > class 선택자 > 태그 선택자 > 전체 선택자 \***
- 개수가 아무리 많아도 우선순위 높은 스타일이 우선 적용

\* 정확히는 CSS 선택자의 명시도(Specificity) 계산법에 따라 결정됨.



```
<div id="box" class="my-box">Hello, World!</div>
```

```
/* 전체 선택자 */
* {
  color: red;
}

/* 태그 선택자 */
div {
  color: blue;
}

/* 클래스 선택자 */
.my-box {
  color: green;
}

/* ID 선택자 */
#box {
  color: purple;
}
```

- 위쪽 HTML 코드에 왼쪽 CSS 코드를 적용하면
- 우선 순위가 가장 높은 ID 선택자의 스타일이 적용 되어 글자가 보라색으로 나타남

Hello, World!



## 4. 텍스트 스타일링



```
.custom-font {  
  font-family: "Times New Roman", Times, serif;  
}
```

- **`font-family`** 속성을 사용하여 사용할 폰트 지정
- 여러 개의 폰트를 순차적으로 지정하여 사용자의 컴퓨터에 폰트가 설치되어 있지 않을 경우 대체 폰트로 사용



```
p{  
    font-size: 16px;  
}
```

- `font-size` 속성을 사용하여 글꼴의 크기 조정
- 크기는 픽셀(px), 포인트(pt), 백분율(%), em 단위 등으로 지정 가능

픽셀 : 디지털 화면에서 가장 작은 단위

포인트 : 1/72인치, 주로 인쇄 매체에서 사용

백분율 : 부모 요소의 폰트 크기에 대한 상대적인 크기

em : 현재 글자의 'font-size'를 기준으로 상대적인 단위 설정



```
.normal-text {  
  font-weight: normal;  
}  
.bold-text {  
  font-weight: bold;  
}  
.numeric-weight {  
  font-weight: 600;  
}
```

- `font-weight` 속성을 사용하여 글꼴의 굵기 조정 가능
- `normal`, `bold`, `bolder`, `lighter` 값을 사용
- 숫자(100, 200, 300, 400, ... , 900)를 사용하기도 함





```
.normal-style {  
    font-style: normal;  
}  
  
.italic-style {  
    font-style: italic;  
}  
  
.oblique-style {  
    font-style: oblique;  
}
```

- `font-style` 속성을 사용하여 텍스트의 스타일 설정 가능
- 일반적으로 `normal` (기본), `italic` (이탤릭체), `oblique` (기울임체) 값을 사용



```
.align-left {  
  text-align: left;  
}  
.align-right {  
  text-align: right;  
}  
.align-center {  
  text-align: center;  
}  
.align-justify {  
  text-align: justify;  
}
```

- ``text-align`` 속성을 사용해 텍스트 정렬
- ``left`` (왼쪽 정렬), ``center`` (가운데 정렬), ``right`` (오른쪽 정렬), ``justify`` (양쪽 정렬) 설정 가능



```
.custom-line-height {  
  line-height: 1.5;  
}  
  
.custom-letter-spacing {  
  letter-spacing: 2px;  
}  
  
.combined {  
  line-height: 2;  
  letter-spacing: 4px;  
}
```

- ``line-height`` 속성을 사용하여 행간 설정
- ``letter-spacing`` 속성을 사용해 글자 간격 조정 가능

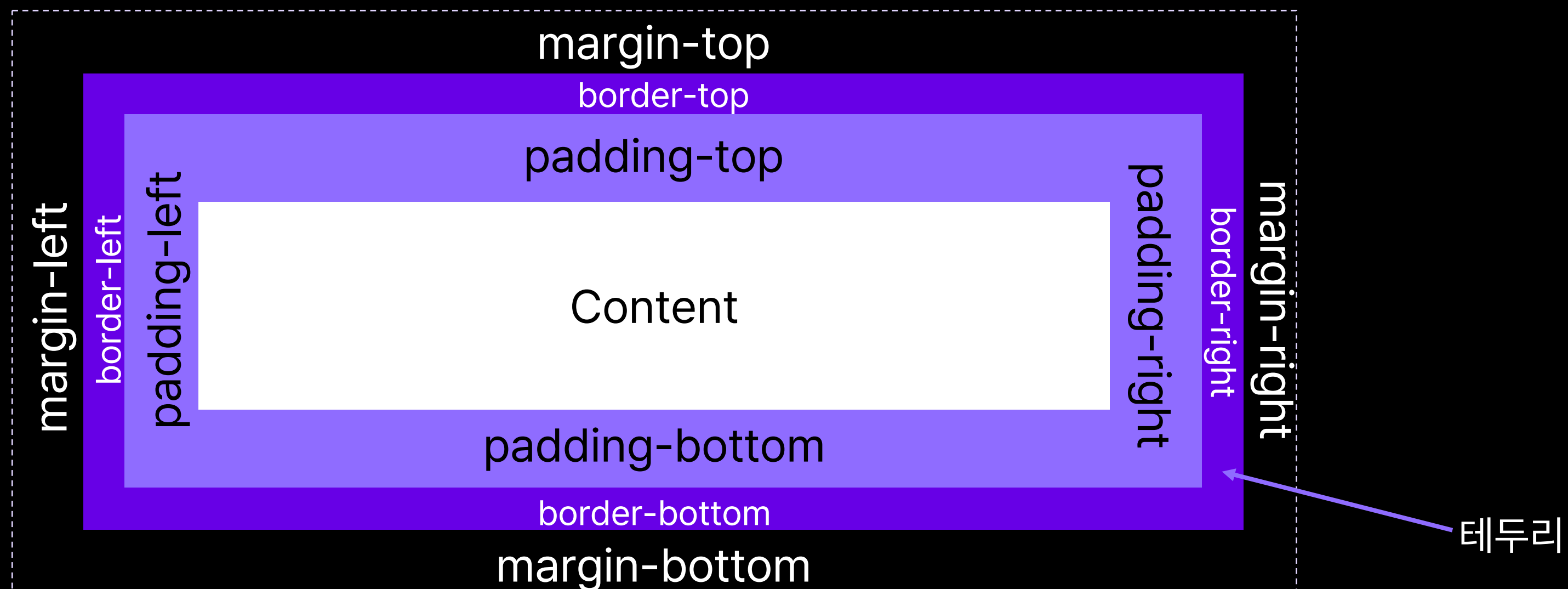
## 5. 박스 모델





## HTML 요소들은 모두 사각형의 박스로 취급, 각 박스의 크기를 결정하는 개념이 박스 모델

- **Content** : 요소의 텍스트나 이미지 등의 실제 내용이 위치하는 영역
- **Padding** : 테두리 안쪽에 위치하는 요소의 내부 영역
- **Border** : 테두리 영역으로 border 프로퍼티 값은 테두리의 두께를 의미
- **Margin** : 테두리(Border) 바깥에 위치하는 요소의 외부 여백 영역





- **Block 요소**

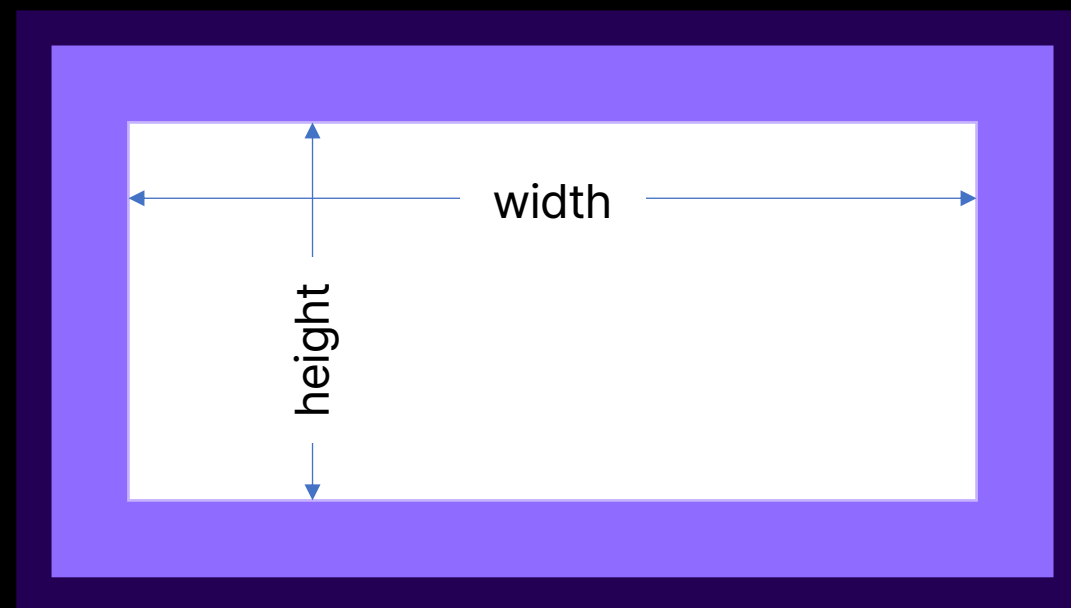
- width : 부모 컨테이너의 전체 너비
- height : 내부 콘텐츠에 의해 결정
- 항상 새로운 줄에서 시작(세로 방향으로 쌓임)
- 상하좌우 margin과 padding 적용 가능

- **Inline 요소**

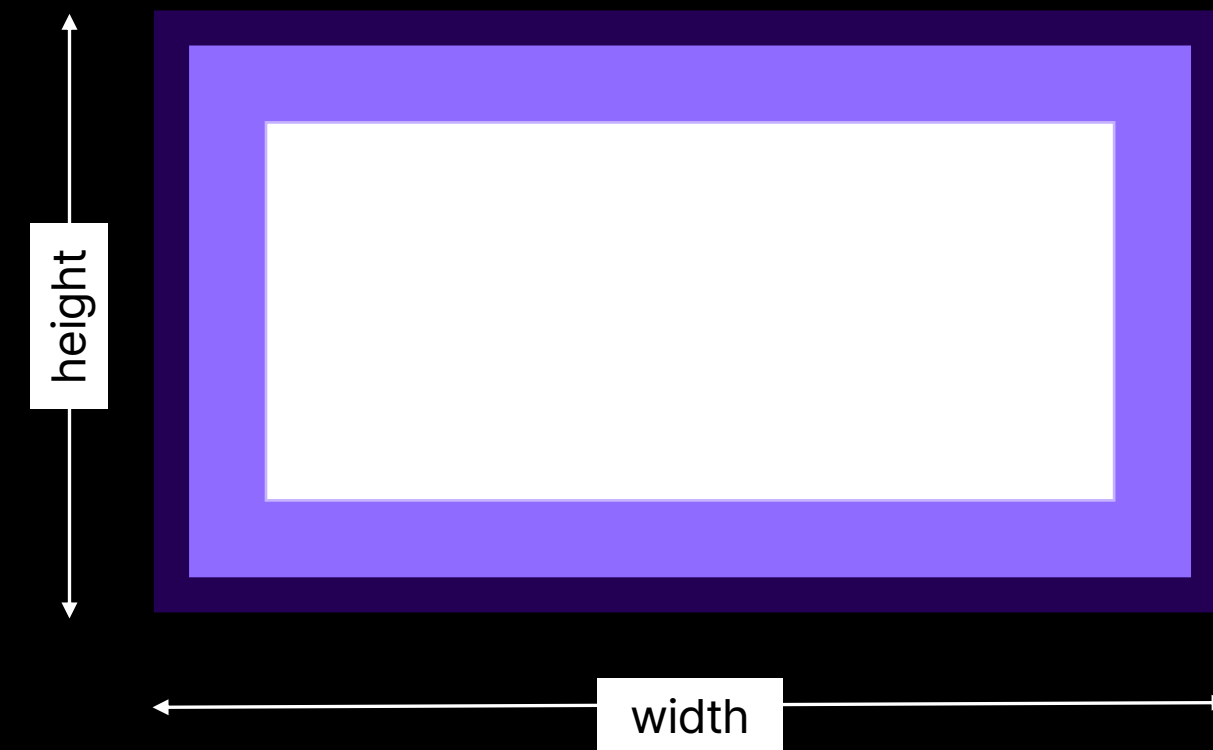
- width와 height가 내부 콘텐츠에 의해 자동으로 결정
- 줄바꿈 하지 않음(가로 방향으로 쌓임)
- padding은 상하좌우 모두 적용되지만, margin은 상하 적용이 되지 않음 (좌우는 적용됨)



- **Box-sizing** : width, height의 대상 영역 변경 가능
- **Content-box** : width, height 프로퍼티 값이 content 영역을 의미 (기본값)
- **Border-box** : width, height 프로퍼티 값이 content 영역, padding, border가 포함된 값을 의미



Content-box를 사용한 경우



Border-box를 사용한 경우



Content의 4개 방향(top, right, bottom, left)에 대하여 지정 가능(padding도 동일)

|               | margin: 25px 50px 75px 100px | margin : 25px 50px 75px | margin : 25px 50px | margin : 25px |
|---------------|------------------------------|-------------------------|--------------------|---------------|
| margin-top    | 25px                         | 25px                    | 25px               | 25px          |
| margin-right  | 50px                         | 50px                    | 50px               | 25px          |
| margin-bottom | 75px                         | 75px                    | 25px               | 25px          |
| margin-left   | 100px                        | 50px                    | 50px               | 25px          |



**border: 1px solid black;**  
line-width line-style color

- `line-width`: border의 굵기
- `line-style`: border 스타일
  - `solid`: 실선
  - `dashed`: 직사각형 점선
  - `dotted`: 원형 점선
- `color`: border 색상



```
<body>
  <div class="box1">첫 번째 박스</div>
  <div class="box2">두 번째 박스</div>
</body>
```

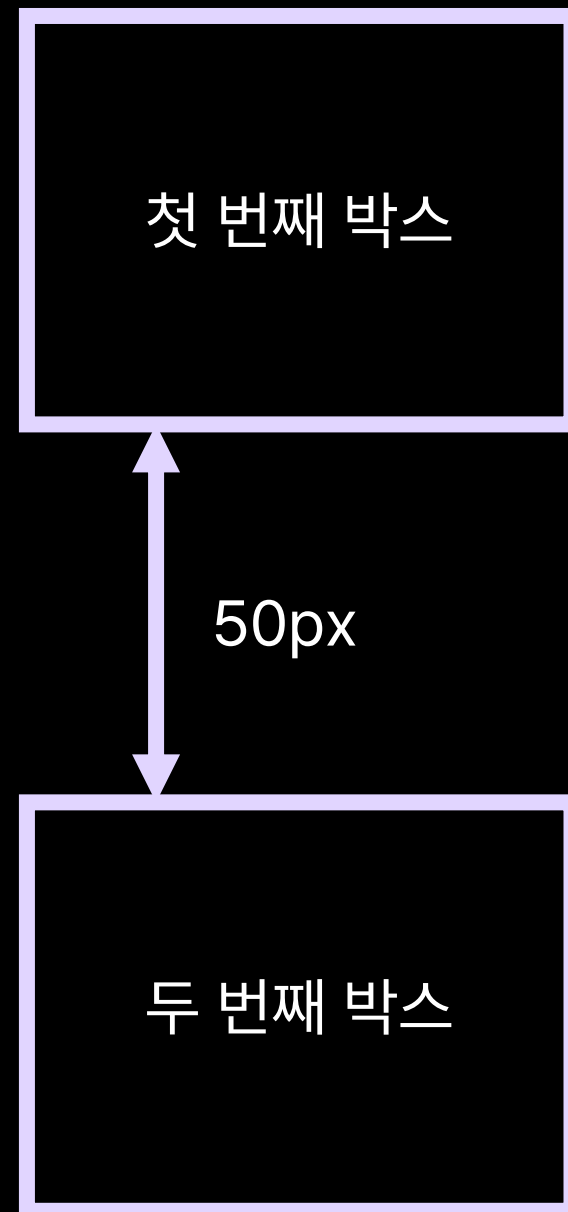
## 형제 요소인 div 요소 생성

```
<style>
  .box1 {
    margin: 20px 0;
  }

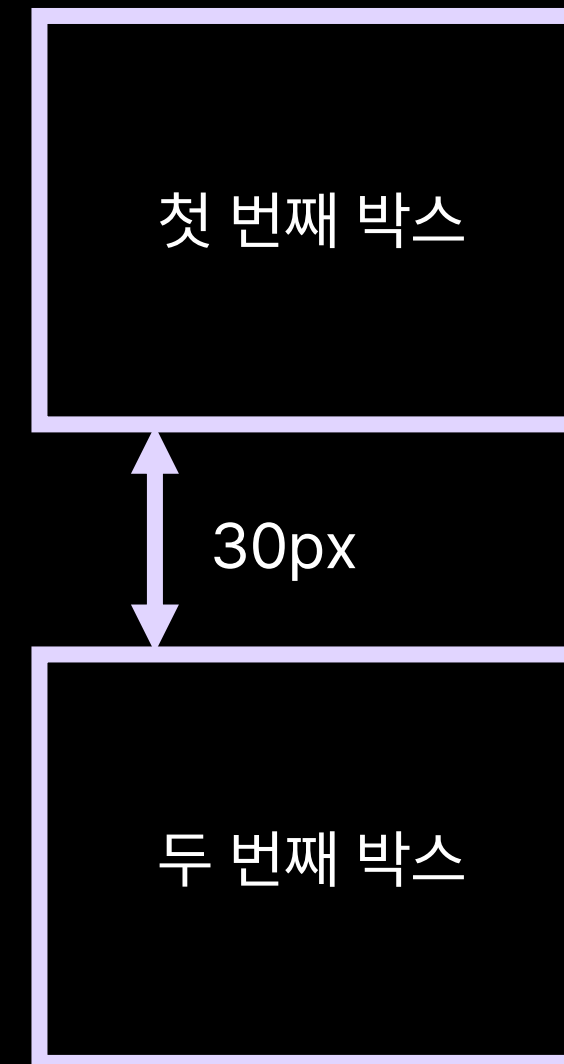
  .box2 {
    margin: 30px 0;
  }
</style>
```

## 형제요소 간의 마진 값 적용

형제 요소 : HTML 문서 내에서 동일한 부모 요소를 가지며 같은 계층에 위치한 요소들을 형제 요소라고 한다.  
왼쪽 코드 예시에서 box1과 box2는 모두 body라는 동일한 부모 요소를 가지며 같은 계층에 위치하므로 형제 요소 관계이다.



예상한 결과



실제 결과





어떤 두 개 이상의 블록 요소의 위 아래의 마진이 겹칠 때,  
제일 큰 크기를 가진 단일 마진으로 결합(상쇄)되는 현상

## 조건

- 인접해 있는 Block 요소 사이에서만 발생함
- 상하단의 마진에 관해서만 발생(좌우로는 발생하지 않음)

## 6. 배경과 그래디언트



- CSS의 **background-color** 속성을 사용하여 요소의 배경 색상 지정 가능
- 색상 이름, HEX 코드, RGB 값을 사용하여 색상 지정 가능

```
/* 배경색을 파란색으로 설정한 예시 */  
.div-background {  
  background-color: blue;  
}
```



- **색 이름** : 미리 정의된 색상 이름을 사용
- **Hex 코드** : 16진수 색상 코드 사용
- **RGB 값** : RGB 색상 모델 사용
- **RGBA 값** : RGB와 더불어 알파 채널 사용
  - A는 투명도  
0.0(완전투명) ~ 1.0(완전 불투명)

| 색 이름   | Hex 코드  | RGB 값          | RGBA 값            | 실제 색상 |
|--------|---------|----------------|-------------------|-------|
| black  | #000000 | rgb(0,0,0)     | rgba(0,0,0,1)     |       |
| red    | #ff0000 | rgb(255,0,0)   | rgba(255,0,0,1)   |       |
| green  | #008000 | rgb(0,128,0)   | rgba(0,128,0,1)   |       |
| blue   | #0000ff | rgb(0,0,255)   | rgba(0,0,255,1)   |       |
| orange | #ffa500 | rgb(255,165,0) | rgba(255,165,0,1) |       |
| lime   | #00ff00 | rgb(0,255,0)   | rgba(0,255,0,1)   |       |





- **background-image** 속성을 사용하여 요소의 배경 이미지 설정 가능
- 이미지 파일 경로를 지정하여 배경으로 사용할 이미지 설정

```
/* 배경 이미지를 설정한 예시 */  
.div-background-image {  
    background-image: url('images/background.jpg');  
}
```



- 두 개 이상의 색상을 서서히 혹은 일정 간격으로 변화시키는 배경효과
- CSS의 선형(**linear-gradient**), 원형(**radial-gradient**) 속성을 사용하여 생성 가능



## • 선형 그라디언트 (linear-gradient)

```
/* 선형 그라디언트를 적용한 예시 */  
.div-linear-gradient {  
  background: linear-gradient(to right, red, blue);  
}
```

이 div의 배경은 선형  
그라디언트입니다.

## • 원형 그라디언트 (radial-gradient)

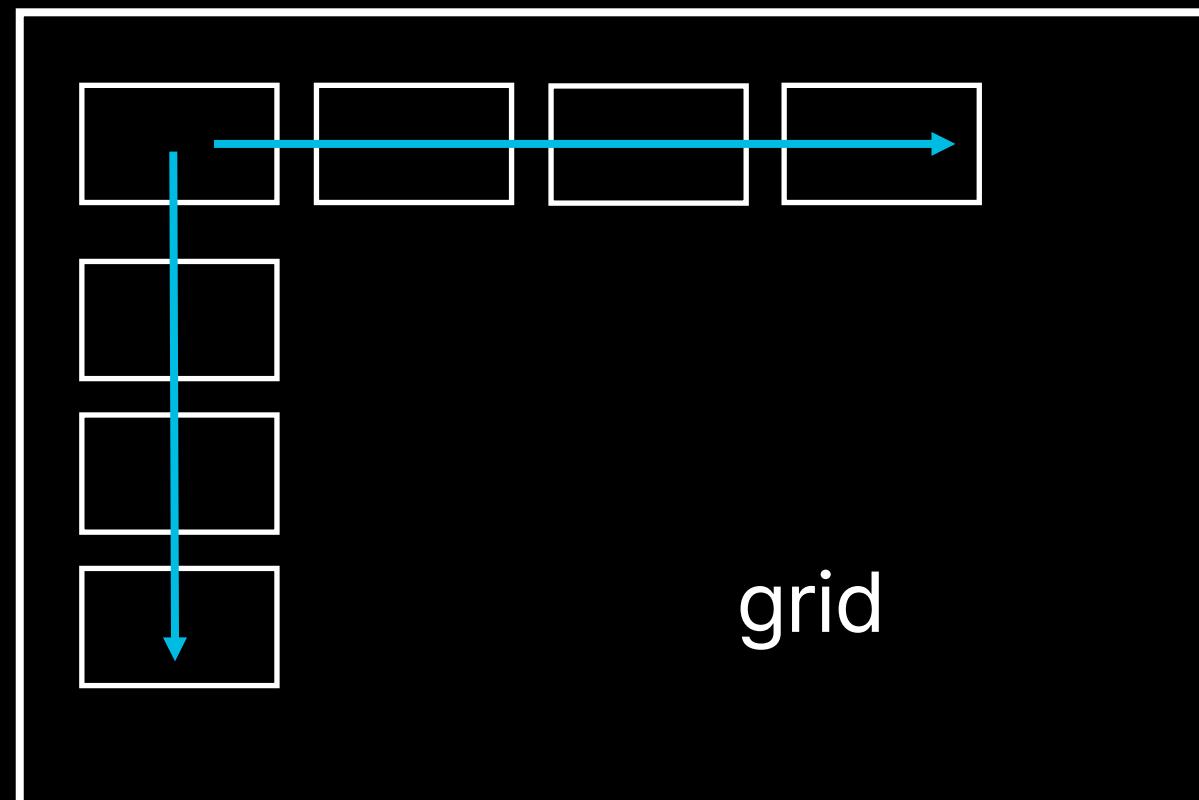
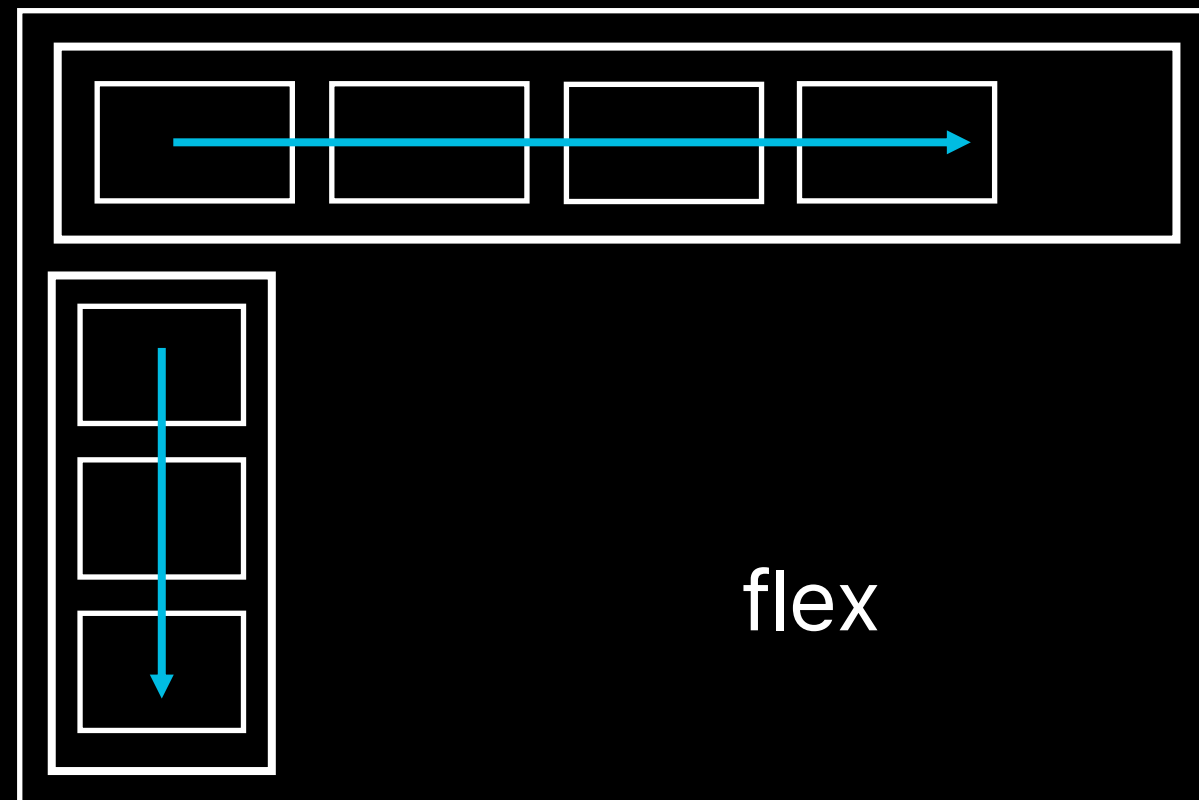
```
/* 원형 그라디언트를 적용한 예시 */  
.div-radial-gradient {  
  background: radial-gradient(circle, yellow, green);  
}
```

이 div의 배경은  
원형 그라디언트  
입니다.



# 7. Flex Layout





## • flex

- 1차원 레이아웃을 위한 도구
- 주 축과 교차 축을 이용해 더 유연한 레이아웃 생성

## • grid

- 2차원 레이아웃을 위한 도구
- 행과 열을 이용해 복잡한 레이아웃 구현 가능

## 1차원 레이아웃을 위한 도구

- **container**와 **item** 2가지 개념으로 구성
- **display: flex;** 설정 후 flex의 다양한 속성을 이용해 자식 요소 배치



# flex 레이아웃을 위한 HTML 구조

HTML/CSS

02 CSS 기초

7. Flex Layout

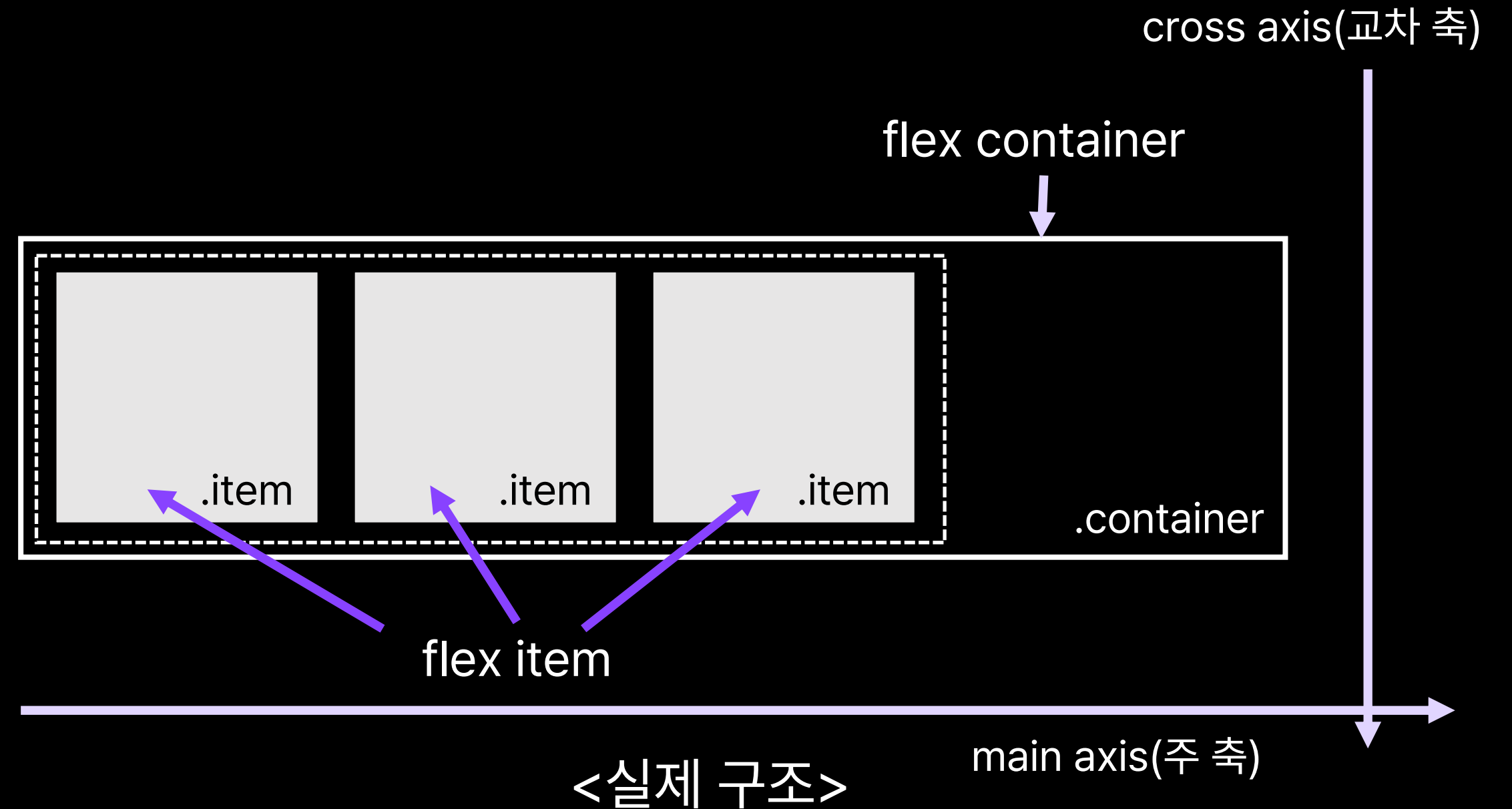
```
<div class="container">
  <div class="item">flex</div>
  <div class="item">elice</div>
  <div class="item">item</div>
</div>
```

flex 레이아웃을 위한 HTML 코드

```
.container {
  display: flex; /* flex 컨테이너로 지정 */
}

.item {
  flex: 1; /* flex 아이템들이
            동일한 공간을 차지하도록 설정 */
}
```

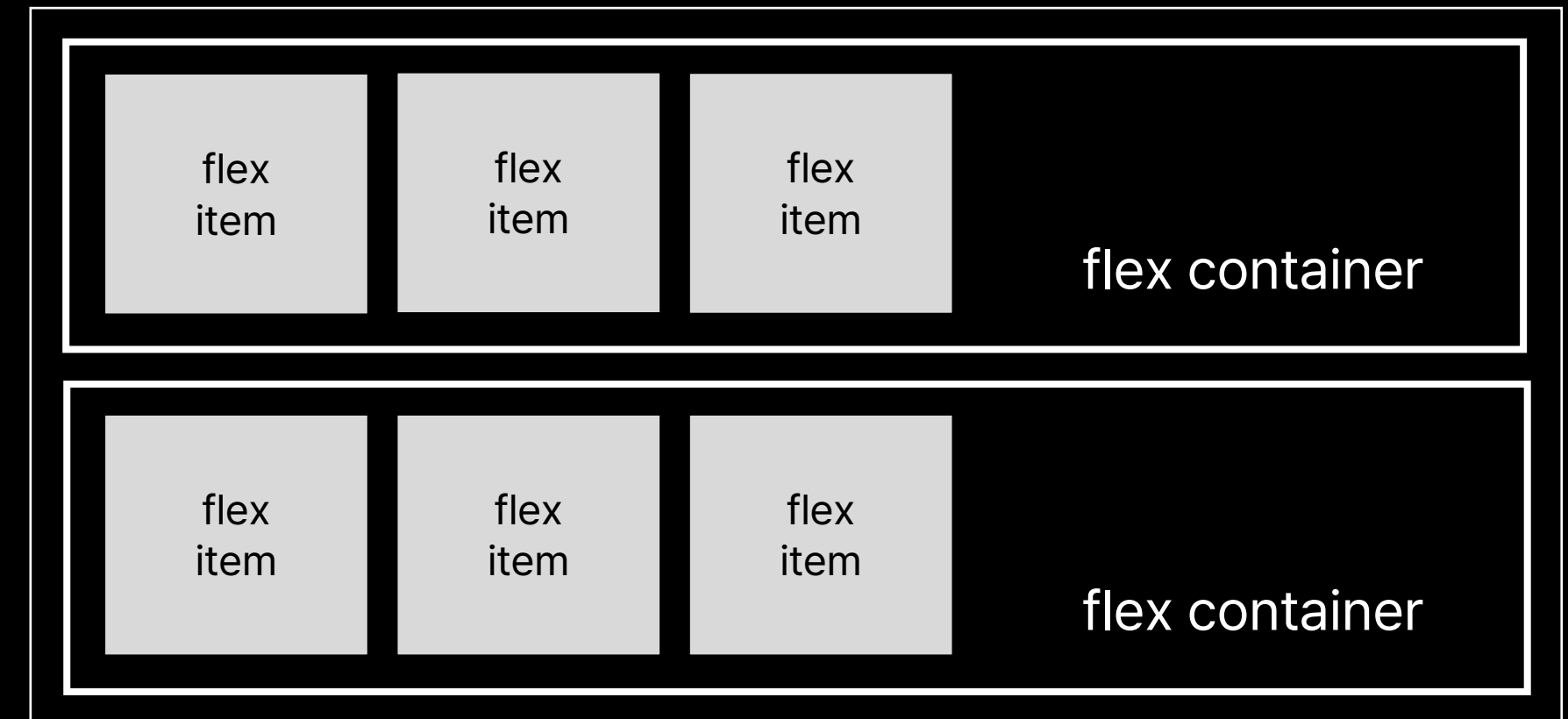
flex 레이아웃을 위한 CSS 코드



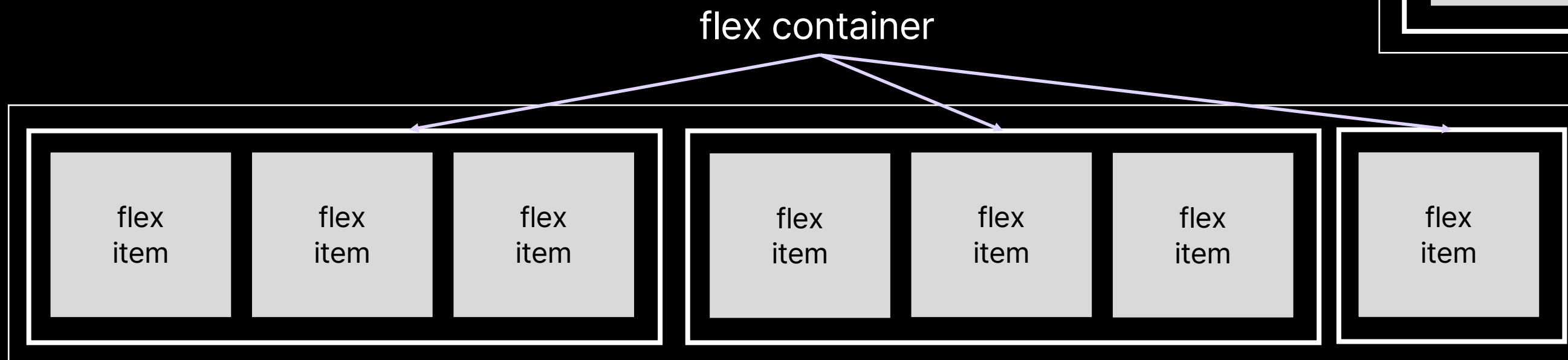
| 속성              | 설명                               |
|-----------------|----------------------------------|
| display         | flex container를 정의               |
| flex-direction  | flex items의 주 축(main-axis) 설정    |
| justify-content | 주 축(main-axis)의 정렬 방법을 설정        |
| align-items     | 교차 축(cross-axis)의 정렬 방법을 설정      |
| flex-wrap       | flex items의 여러 줄 묶음(줄 바꿈) 설정     |
| flex-flow       | flex-direction과 flex-wrap의 단축 속성 |



| 속성과 값                | 의미                           |
|----------------------|------------------------------|
| display: flex        | Block 특성의 flex container 정의  |
| display: inline-flex | Inline 특성의 flex container 정의 |

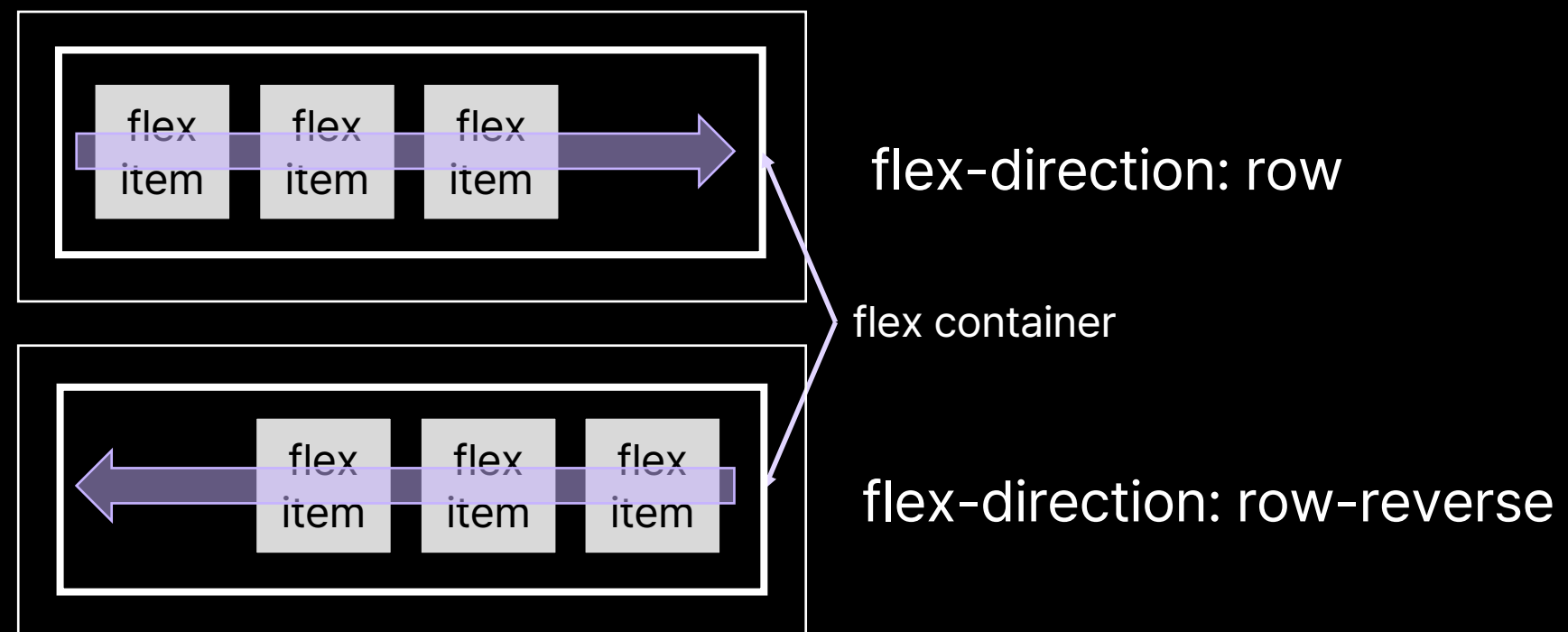


display: flex의 예시

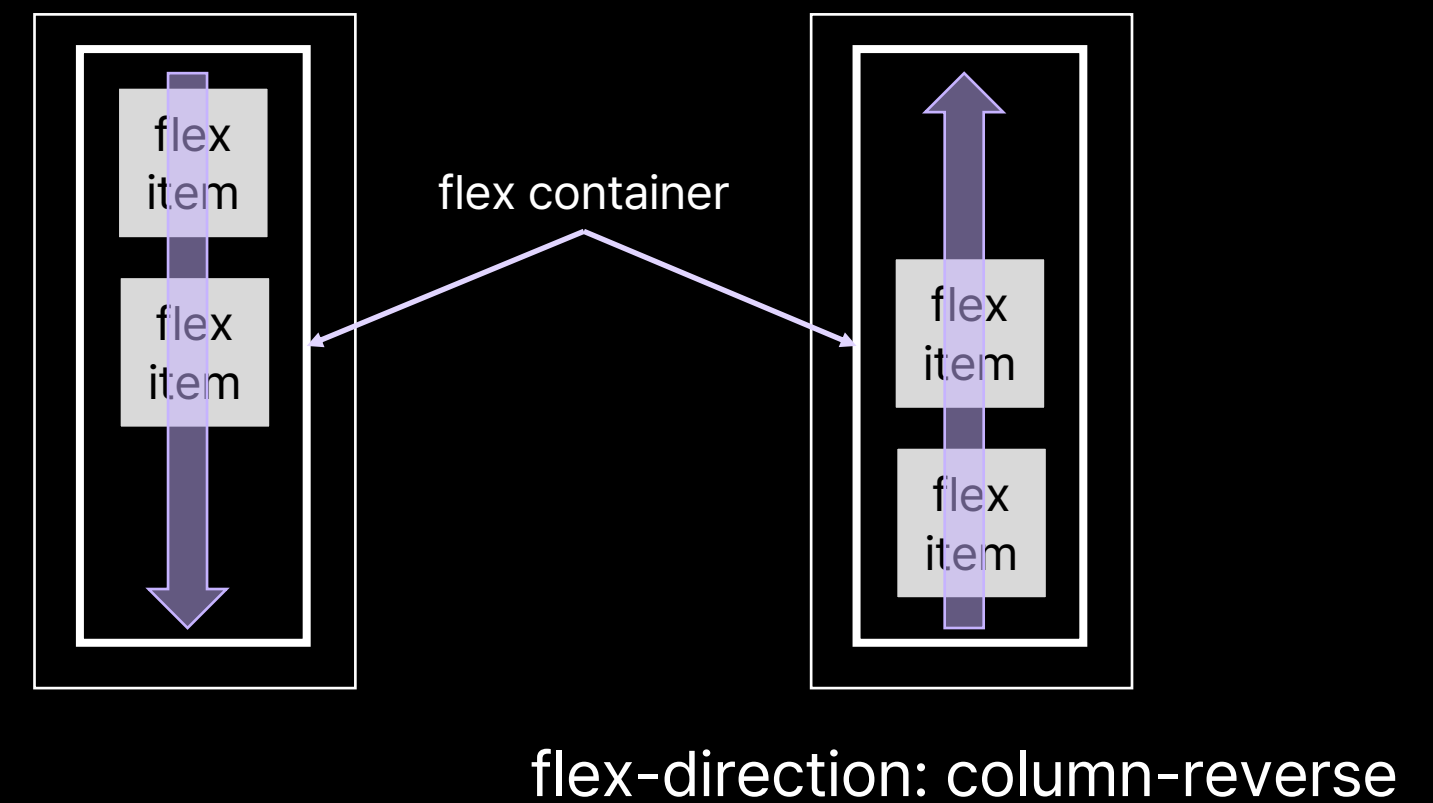


display: inline-flex의 예시

| 속성과 값                          | 의미                                      |
|--------------------------------|-----------------------------------------|
| flex-direction: row (기본값)      | Items를 수평축(왼쪽에서 오른쪽)으로 표시 (주 축이 오른쪽 방향) |
| flex-direction: row-reverse    | Items를 row의 반대로 표시 (주 축이 왼쪽 방향)         |
| flex-direction: column         | Items를 수직축(위에서 아래)으로 표시 (주 축이 아래쪽 방향)   |
| flex-direction: column-reverse | Items를 column의 반대 축으로 표시 (주 축이 위쪽 방향)   |

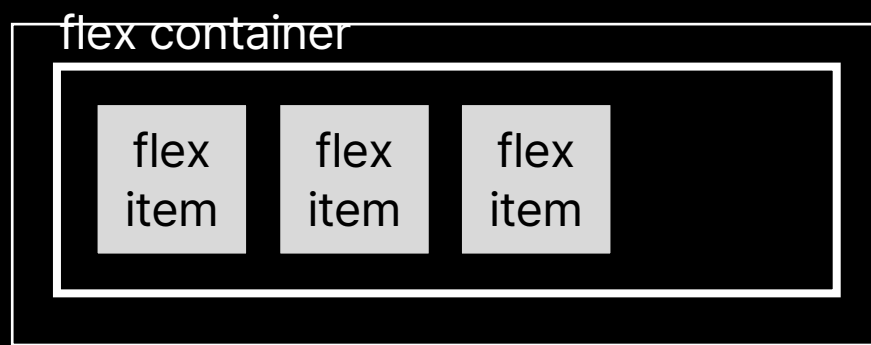


flex-direction: column

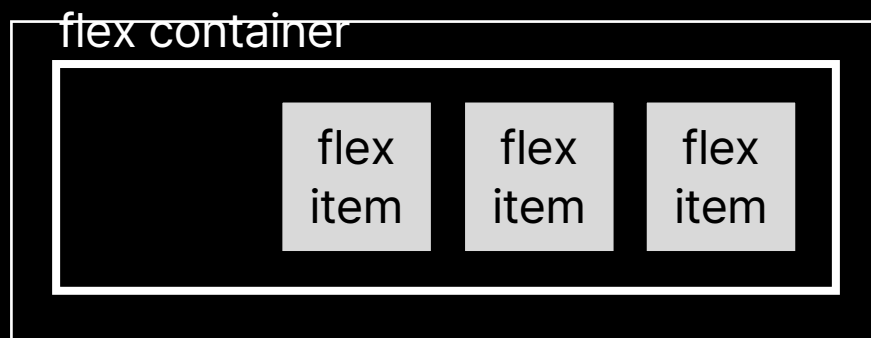


## • 주축(main-axis)의 정렬 방법을 설정

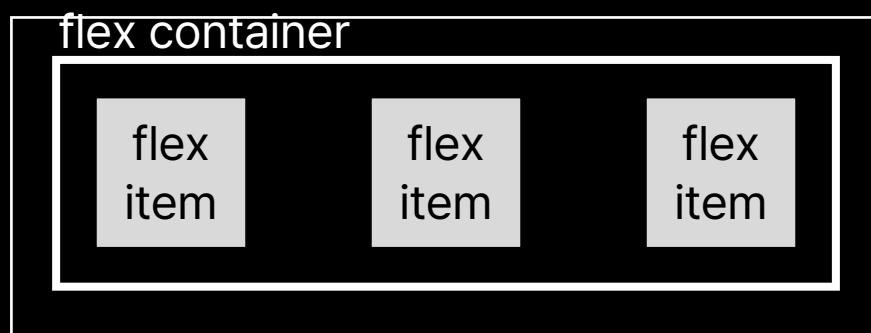
### flex-direction: row



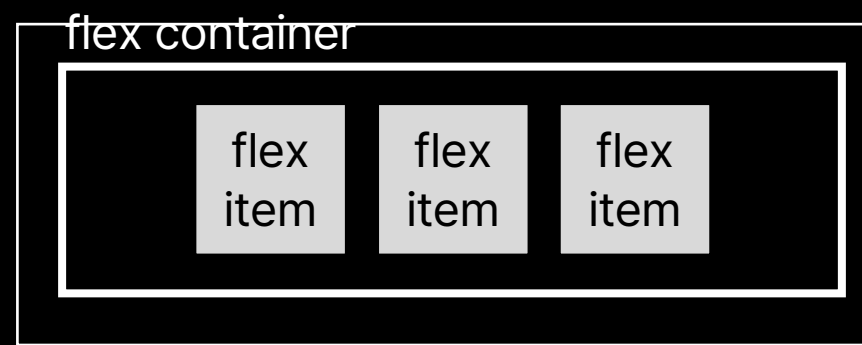
justify-content: flex-start



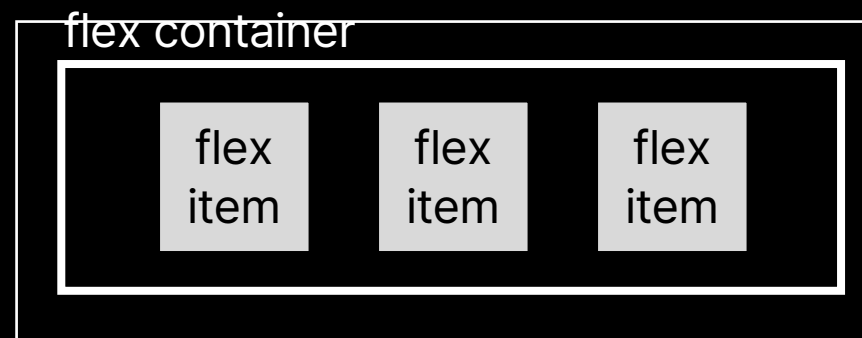
justify-content: flex-end



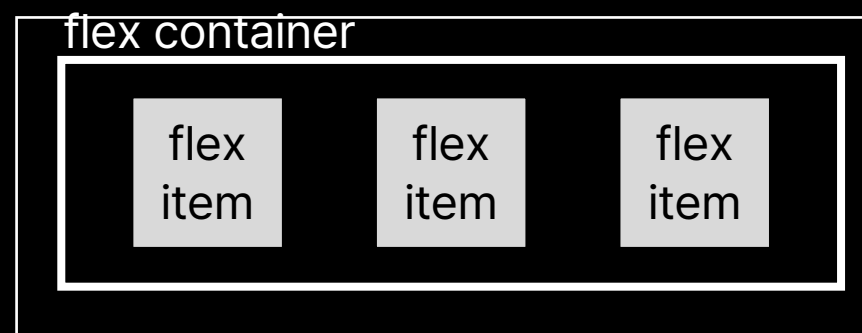
justify-content: space-between



justify-content: center



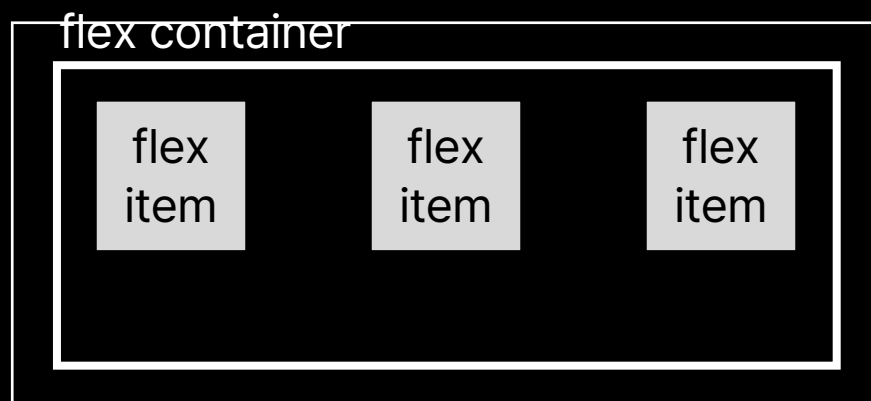
justify-content: space-evenly



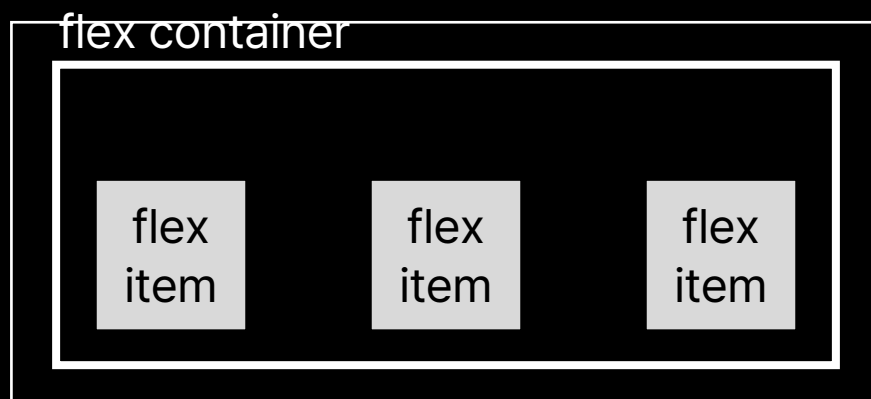
justify-content: space-around

## • 교차 축(**cross-axis**)의 정렬 방법을 설정

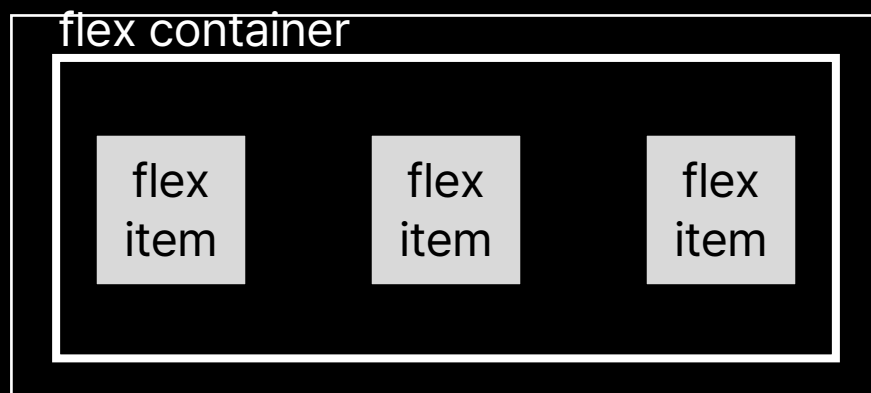
**flex-direction: row**  
**justify-content: space-between**



align-items: flex-start



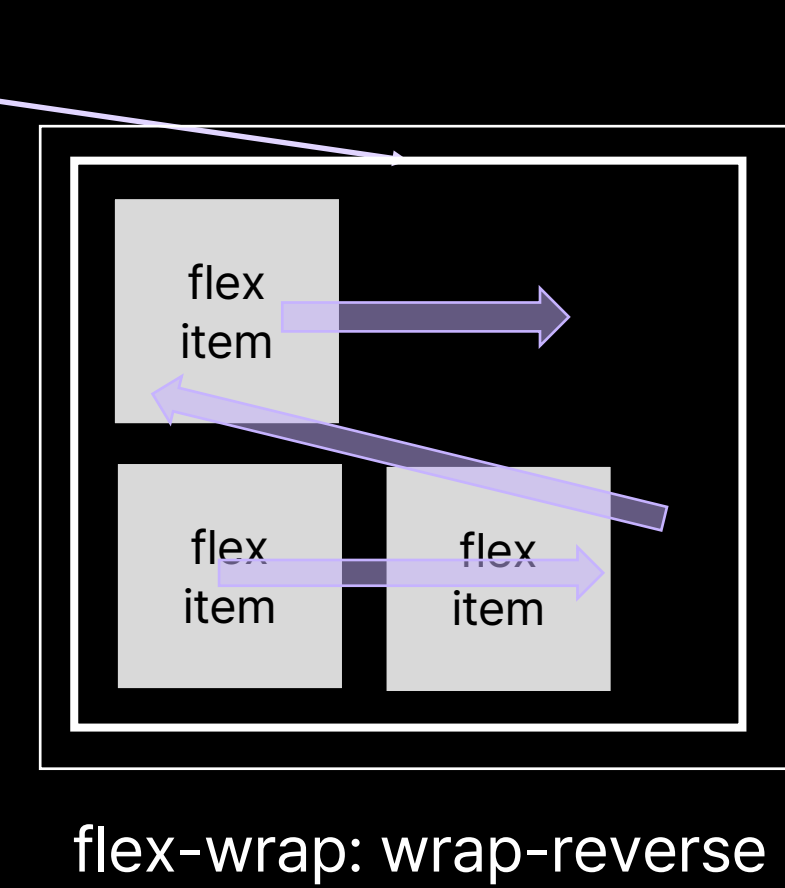
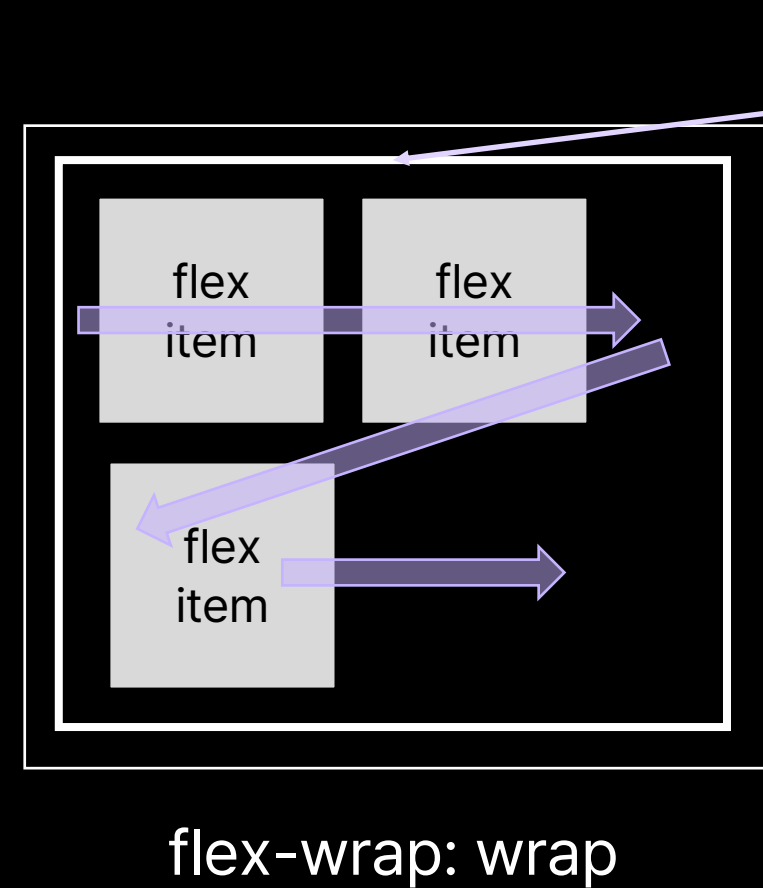
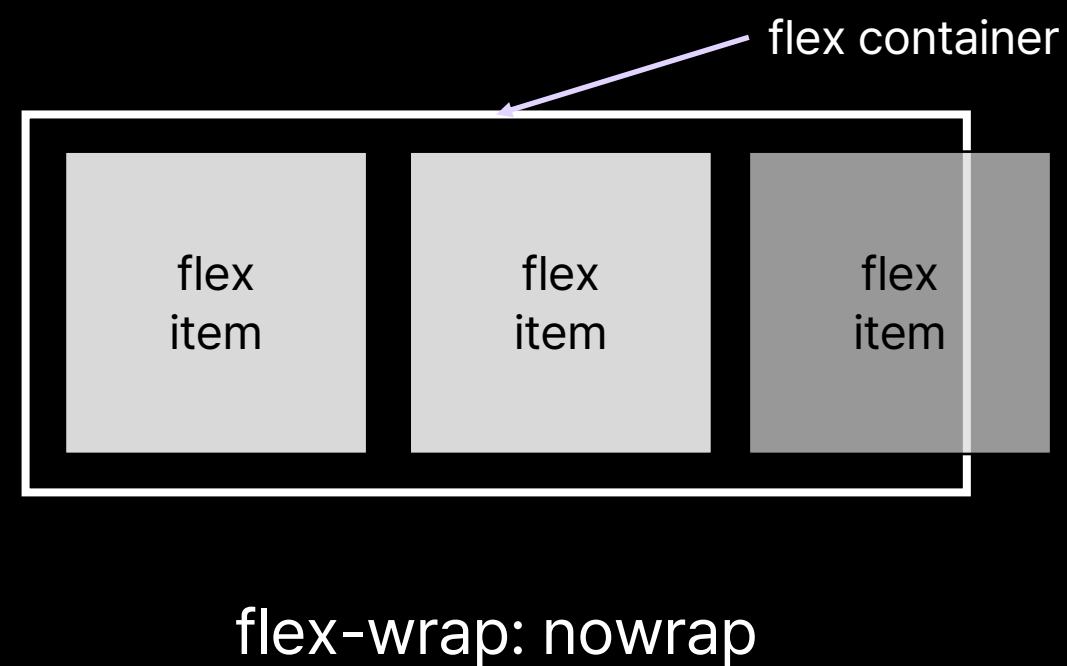
align-items: flex-end



align-items: center



| 속성과 값                   | 의미                            |
|-------------------------|-------------------------------|
| flex-wrap: nowrap (기본값) | 모든 Items를 여러 줄로 묶지 않음(1줄에 표시) |
| flex-wrap: wrap         | Items를 여러 줄로 묶음               |
| flex-wrap: wrap-reverse | Items를 wrap의 역방향으로 여러 줄로 묶음   |





## flex-direction과 flex-wrap을 한 번에 지정하는 속성

```
.container {  
  flex-flow: row wrap;  
  /* 아래의 두 속성을 한 번에 지정 */  
  /* flex-direction: row; */  
  /* flex-wrap: wrap; */  
}
```



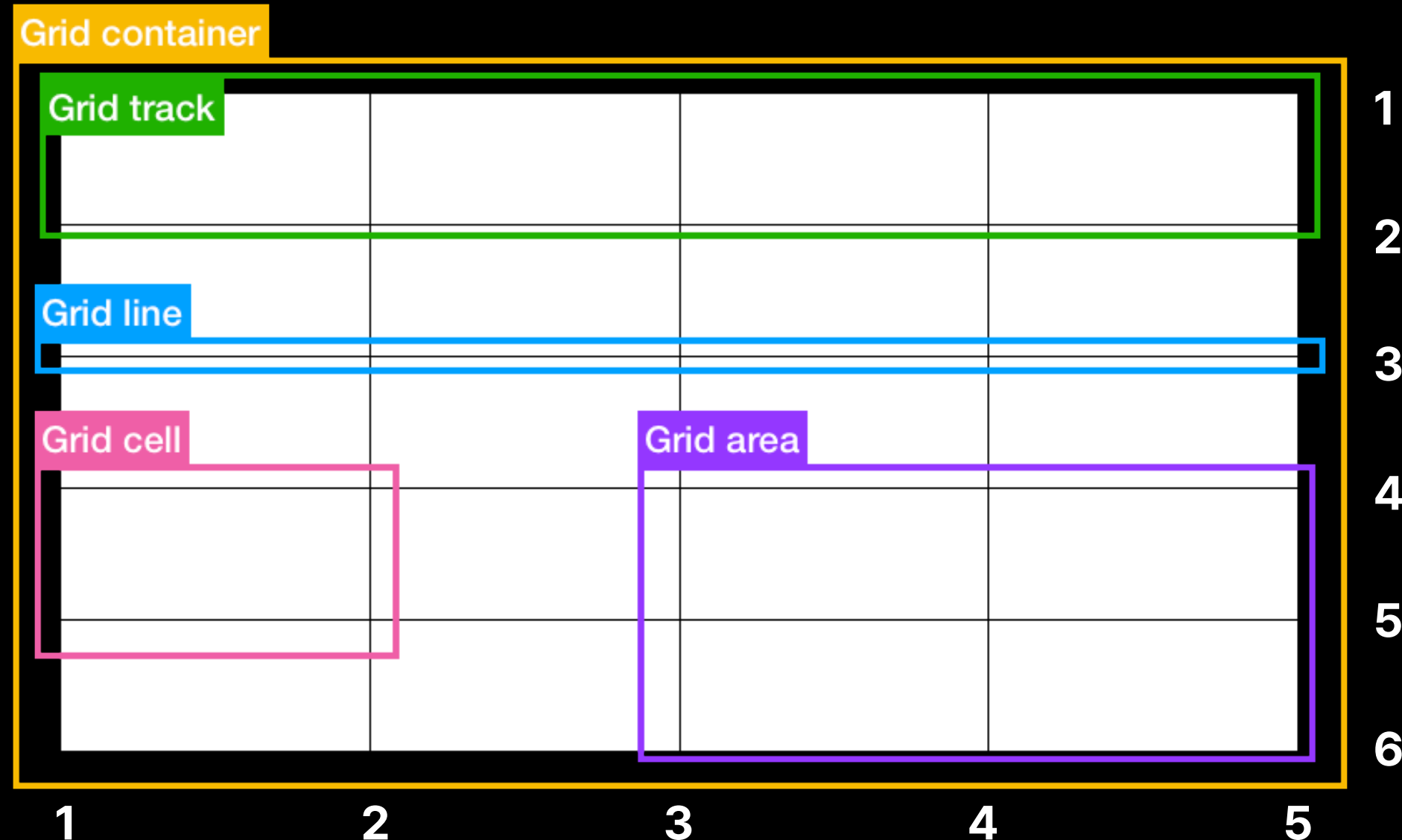
| 속성          | 설명                                 | 기본값     | 속성값                          | 설명               |
|-------------|------------------------------------|---------|------------------------------|------------------|
| order       | flex item의 순서를 결정                  | 0       | 숫자                           | 숫자 작을수록 앞 순서     |
| flex-grow   | flex item의 증가 너비 비율을 설정            | 0       | 숫자                           | 증가 비율            |
| flex-shrink | flex item의 감소 너비 비율을 설정            | 1       | 숫자                           | 감소 비율            |
| flex-basis  | flex item의 (공간 배분 전) 기본 너비 설정      | auto    | 단위                           | px, em, rem, ... |
| flex        | flex-grow, flex-shrink, flex-basis |         |                              |                  |
| align-self  | 교차 축(cross-axis)에서 item의 정렬 방법을 설정 | stretch | flex-start, flex-end, center | 아이템 정렬           |

# 8. Grid Layout



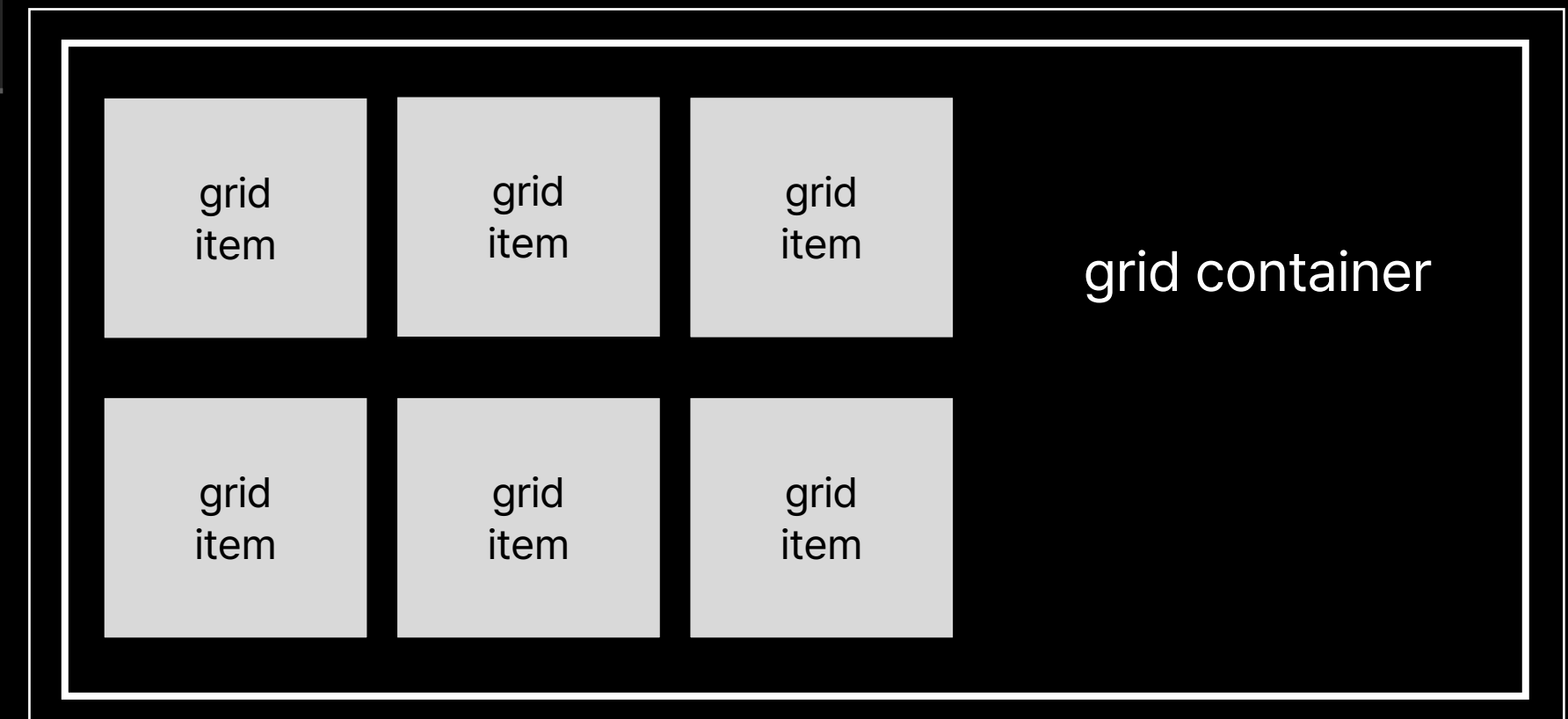
## 2차원 레이아웃을 위한 도구

- **행('row')**과 **열('column')**을 정의하고 이를 이용해 자유자재로 요소 배치가능
- **display: grid;** 설정 후 grid의 다양한 속성을 이용해 자식 요소 배치

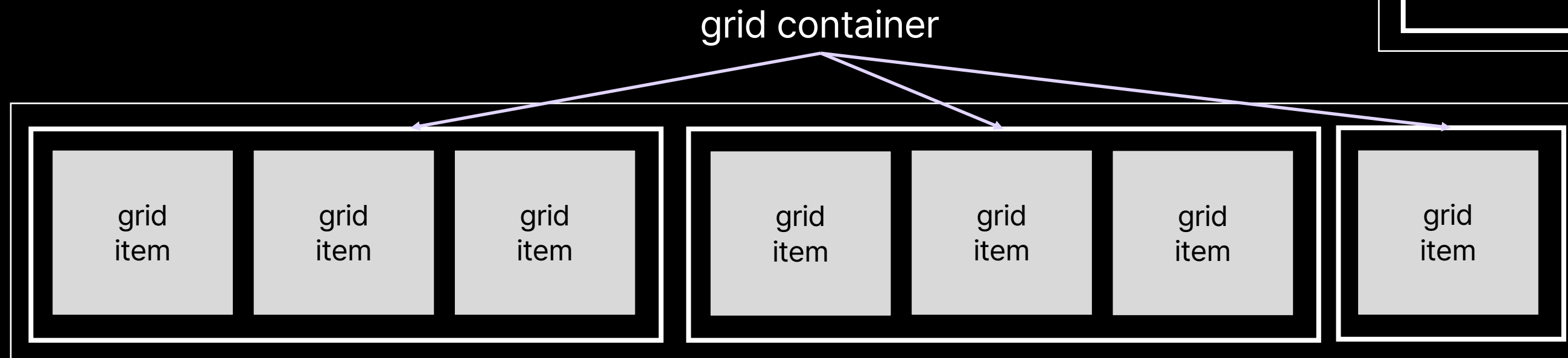


- **Grid container:** 전체 Grid Layout을 감싸는 역할 수행
- **Grid item:** Grid container에 속해 있는 하위 DOM 요소
- **Grid track:** Grid Layout에 존재하는 행 또는 열 의미
- **Grid line:** Grid track을 구분하는 선을 의미
- **Grid cell:** Grid Layout에서 가장 작은 단위요소. 테이블의 셀과 유사
- **Grid area:** 다수의 Grid cell로 이루어진 영역

| 속성과 값                | 의미                           |
|----------------------|------------------------------|
| display: grid        | Block 특성의 grid container 정의  |
| display: inline-grid | Inline 특성의 grid container 정의 |



display: grid의 예시

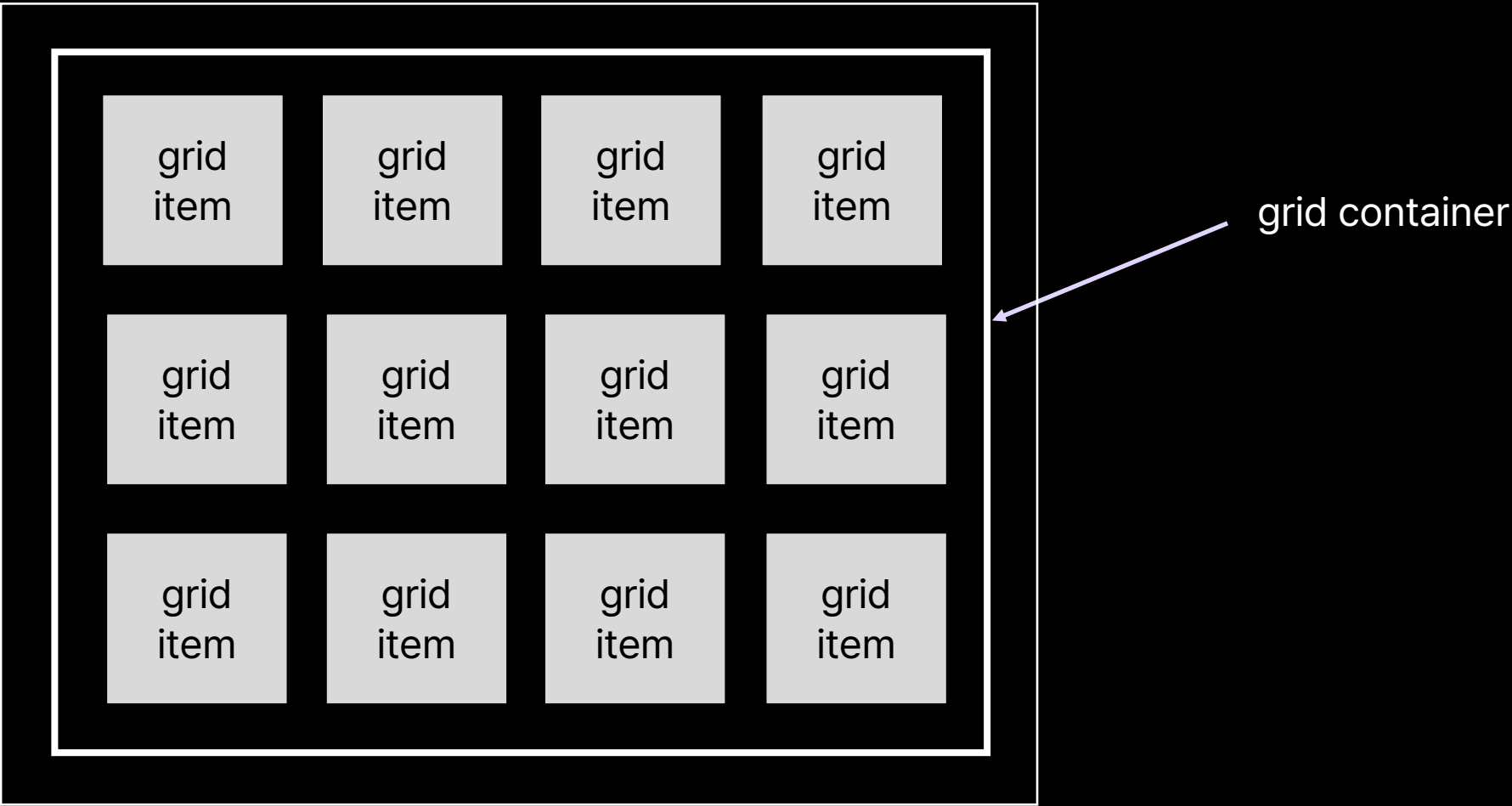


display: inline-grid의 예시



| 속성                    | 의미                                               |
|-----------------------|--------------------------------------------------|
| grid-template-rows    | row의 배치 결정                                       |
| grid-template-columns | column의 배치 결정                                    |
| grid-auto-columns     | grid-template-columns의 통제를 벗어난 위치에 있는 트랙의 크기를 지정 |
| grid-auto-rows        | grid-template-rows의 통제를 벗어난 위치에 있는 트랙의 크기를 지정    |

```
.grid_container {  
  grid-template-columns: 1fr 1fr 1fr 1fr;  
  /* fr: 공간을 비례적으로 분배함 */  
}
```



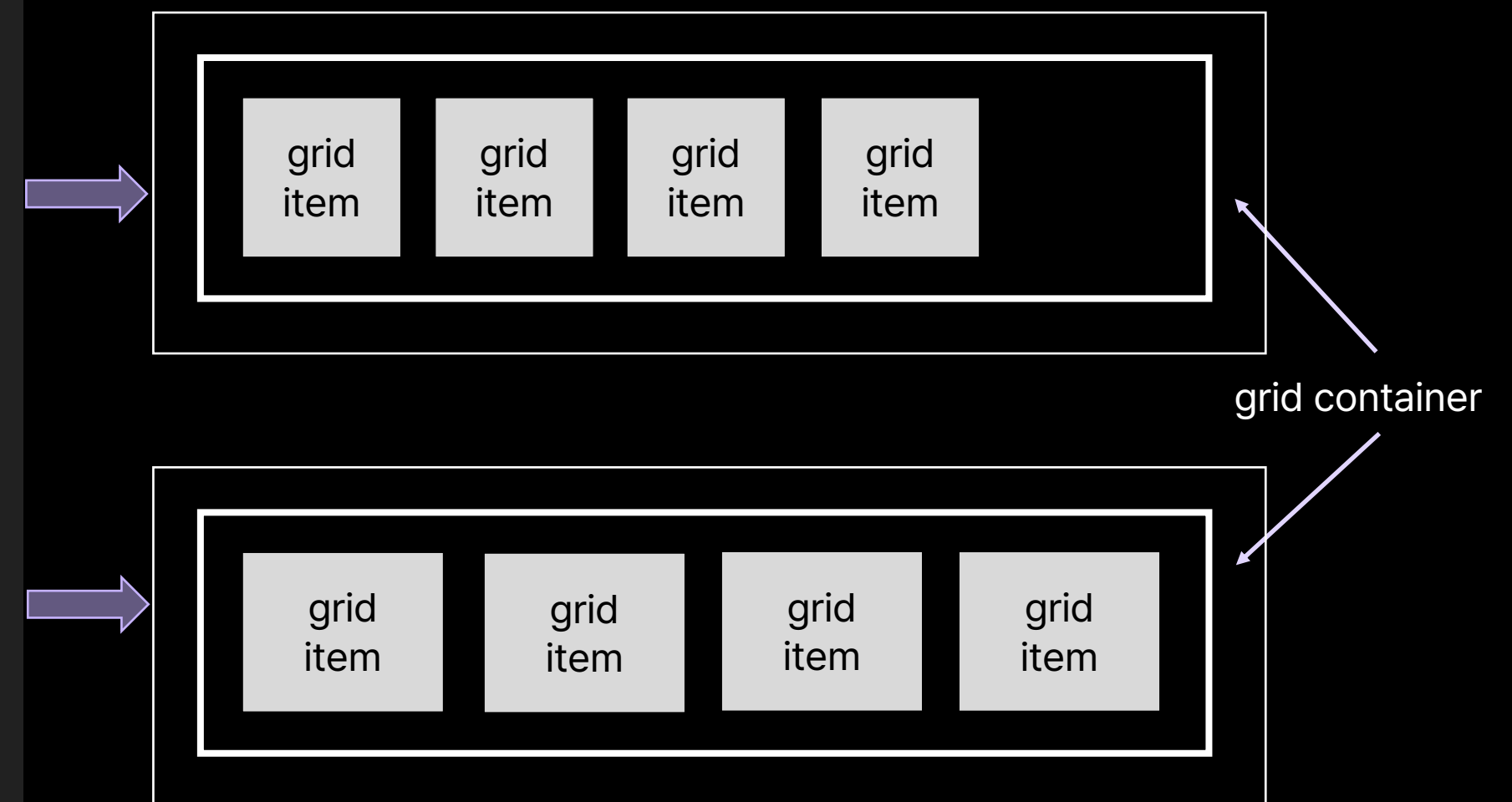




| 값                   | 의미                                    |
|---------------------|---------------------------------------|
| repeat(반복횟수, 반복값)   | 반복되는 값 자동으로 처리하는 함수                   |
| minmax(최소값, 최대값)    | 최소, 최대한의 폭 지정하는 함수                    |
| auto-fill, auto-fit | column의 개수를 미리 정하지 않고 설정된 너비만큼의 셀을 채움 |

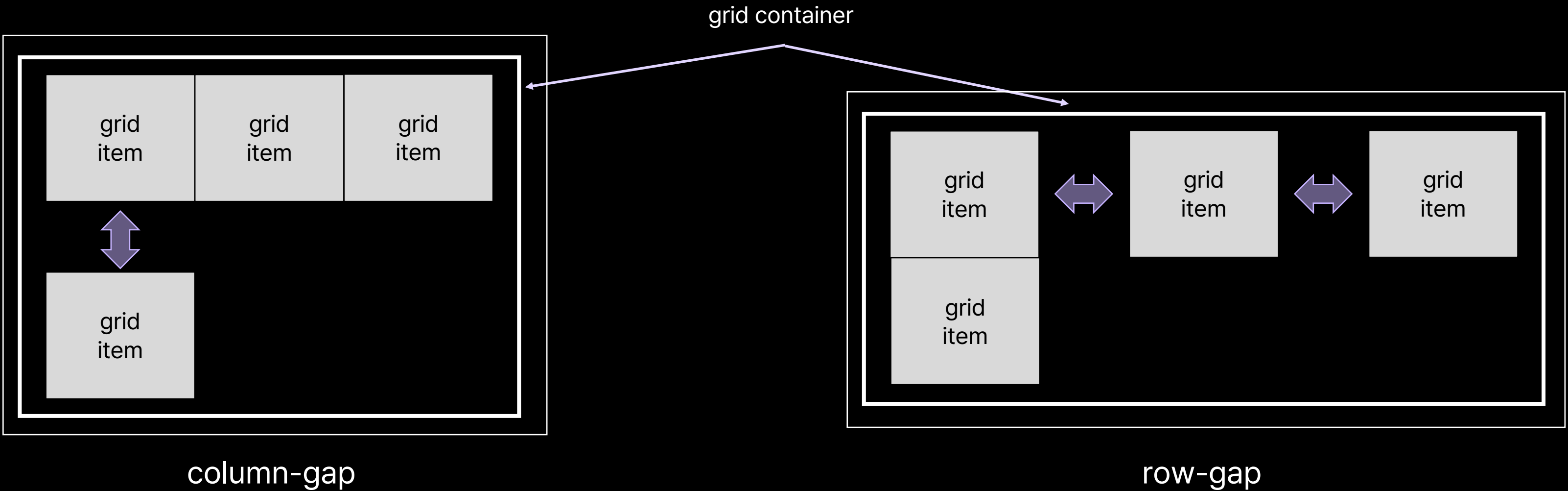
```
.grid_container {  
  grid-template-columns: repeat(auto-fill, minmax(20%, auto));  
  /* fr: 공간을 비례적으로 분배함 */  
  /* 20% -> 1개의 row에는 5개의 셀이 들어갑니다. */  
}
```

```
.grid_container {  
  grid-template-columns: repeat(auto-fit, minmax(20%, auto));  
  /* fr: 공간을 비례적으로 분배함 */  
  /* auto-fit: 남은 공간을 채웁니다. */  
}
```





| 속성         | 의미                           |
|------------|------------------------------|
| row-gap    | grid cell 사이의 row 간격을 설정함    |
| column-gap | grid cell 사이의 column 간격을 설정함 |
| gap        | grid cell 사이의 간격을 설정함        |





# 각 셀의 영역 지정

## 속성

grid-column-start

grid-column-end

grid-column

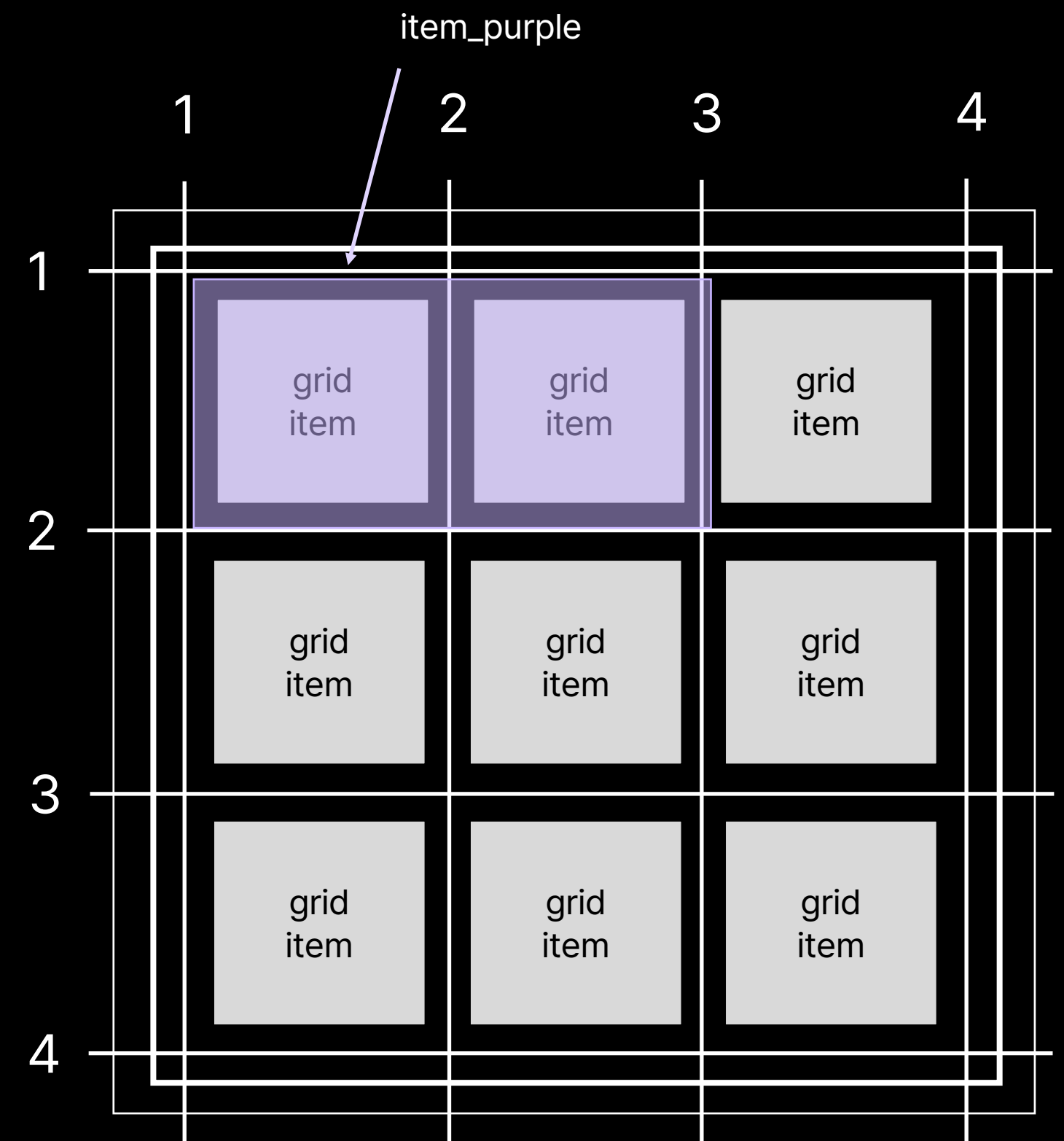
grid-row-start

grid-row-end

grid-row

```
.item_purple {  
  grid-column: 1/3;  
  grid-row: 1/2;  
}  
  
.item_purple {  
  grid-column-start: 1;  
  grid-column-end: 3;  
  grid-row-start: 1;  
  grid-row-end: 2;  
}
```

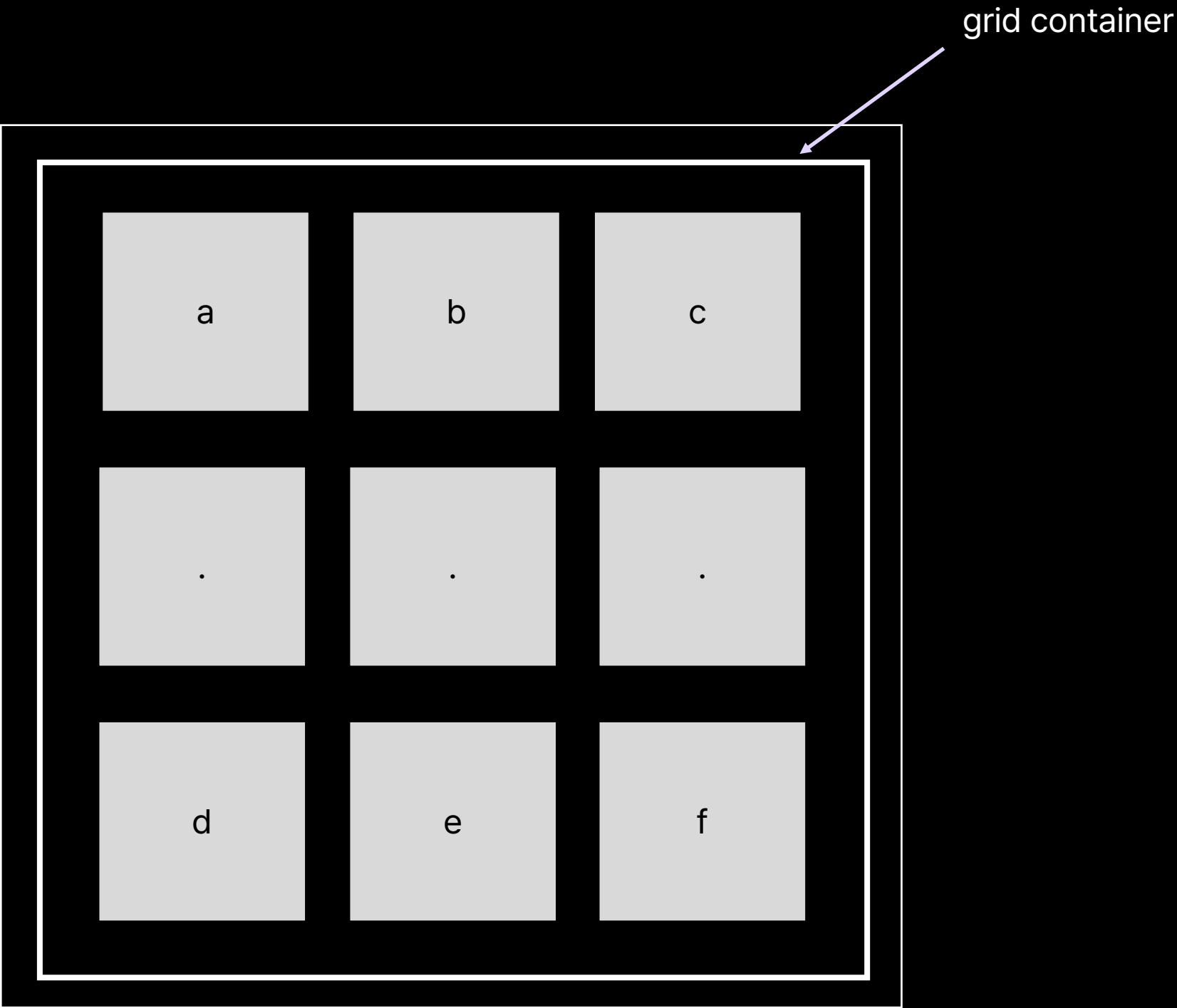
*/\* 위의 두 코드는 모두 보라색 영역을 지정합니다. \*/*





| 속성                  | 의미                                           |
|---------------------|----------------------------------------------|
| grid-template-areas | 각 영역(Grid Area)에 이름을 붙이고, 그 이름을 이용해서 배치하는 방법 |

```
.grid_container {  
  grid-template-areas:  
    "a b c"  
    ". . ."  
    "d e f";  
}  
  
/* ex) header 아이템이 차지하는 영역에 header 라고 이름 붙이기*/  
.header { grid-area: header; }
```





| 속성                                            | 의미                          |
|-----------------------------------------------|-----------------------------|
| grid-auto-flow                                | 아이템 자동 배치되도록 결정             |
| order                                         | 각 아이템들의 <b>시각적</b> 나열 순서 지정 |
| align-items, justify-items, place-items       | 아이템들 세로, 가로, 세로+가로 정렬       |
| align-content, justify-content, place-content | 아이템 그룹 세로, 가로, 세로+가로 정렬     |
| align-self, justify-self, place-self          | 아이템 세로, 가로, 세로+가로 정렬        |

# 9. Position



## 웹 페이지 디자인의 핵심

- 웹 페이지의 첫인상 결정
- 사용자 경험(UX)과 직접 연결
- 디자인의 목적과 내용에 따른 레이아웃 선택의 중요성
- **Display** 속성, **Position** 등을 이용해 구현 가능

## 화면에 어떻게 드러나게 할 것인지에 관한 태그

- **None** : 화면에서 사라지고 공간을 차지하지도 않음
- **Block** : 새로운 줄에서 시작, 가능한 모든 가로 공간 차지
- **Inline** : 새 줄을 시작하지 않고, 필요한 만큼의 공간만 차지
- **Inline-block** : inline처럼 새 줄을 시작하지 않지만, block처럼 높이와 너비 지정 가능
- **+ Flex, Grid**



## HTML 문서 상에서 요소가 배치되는 방식 결정

- **Relative** : positioning의 기준이 되는 요소로 지정
  - **Absolute** : 가장 가까운 relative 조상 요소에 대해 상대적으로 이동
  - **Fixed** : 뷰포트에 대해 상대적으로 이동
  - **Sticky** : 스크롤 위치에 따라 상대적, 절대적으로 동작
- 
- Position을 사용하면 **top/right/bottom/left** 속성으로 위치 조정 가능

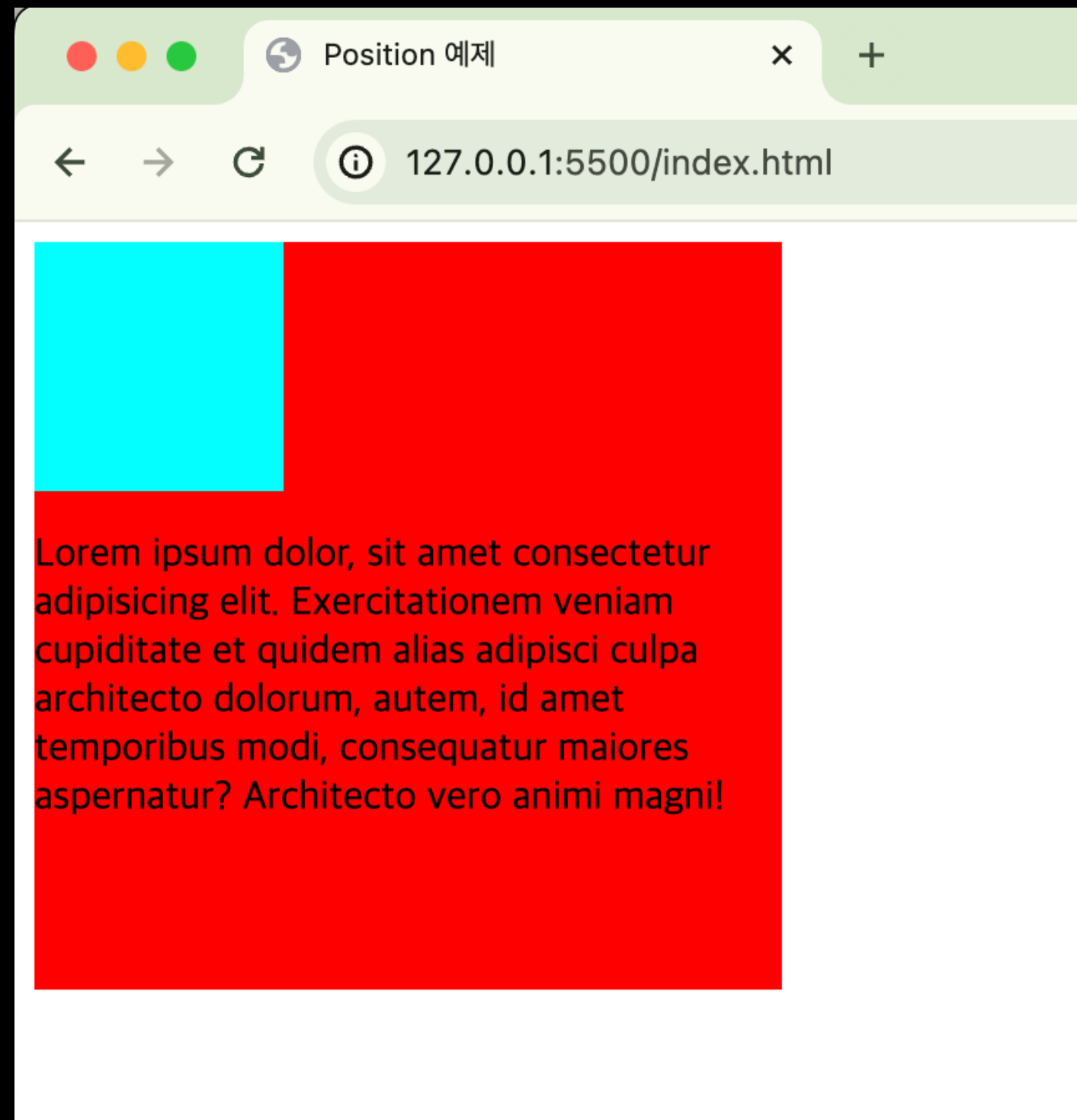


# Position Relative, Absolute

HTML/CSS

02 CSS 기초

9. Position



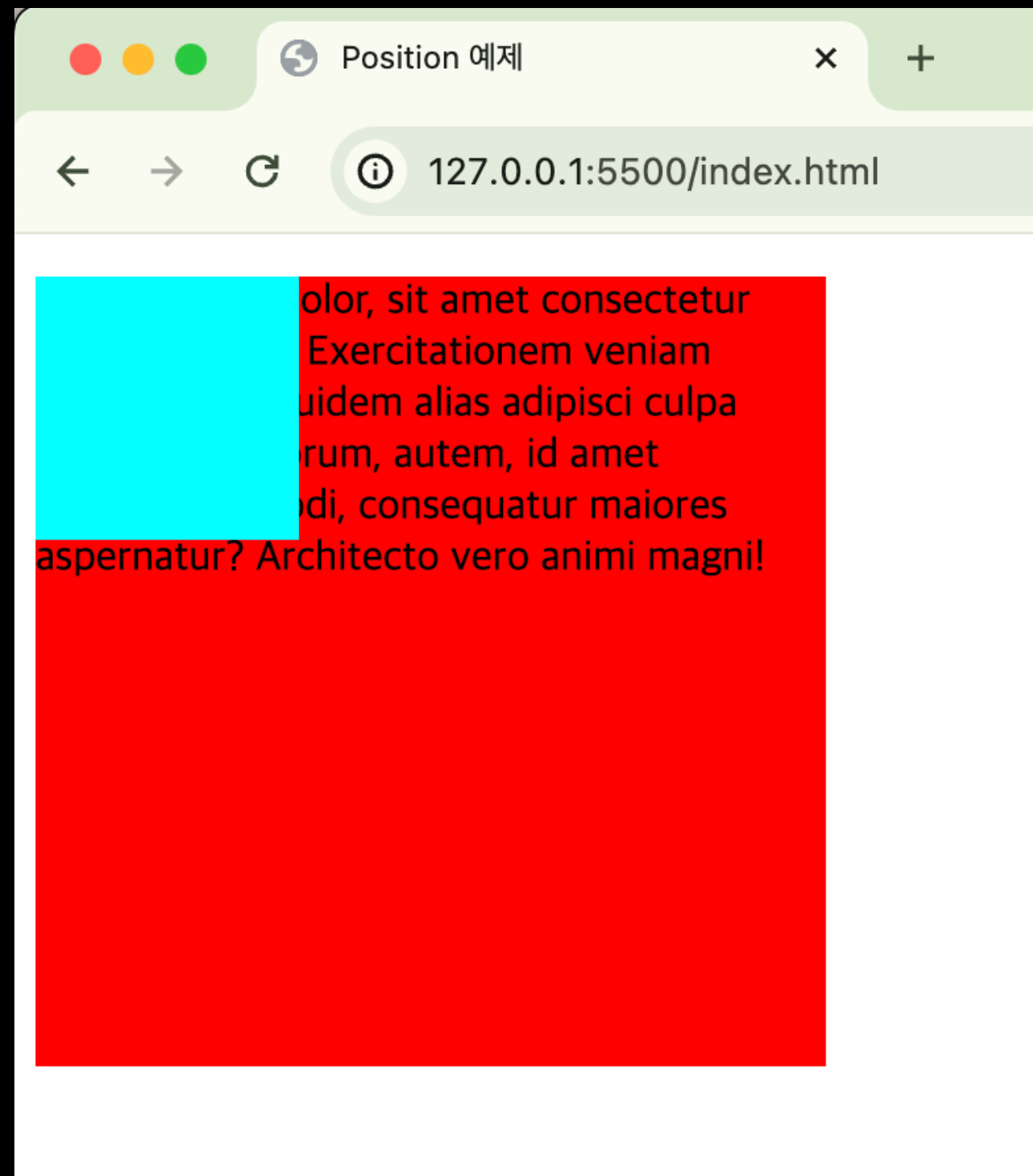
```
<div id="parent">
  <div id="child"></div>
  <p>
    Lorem ipsum dolor, sit amet consectetur adipisicing elit. Exercitationem
    veniam cupiditate et quidem alias adipisci culpa architecto dolorum,
    autem, id amet temporibus modi, consequatur maiores aspernatur?
    Architecto vero animi magni!
  </p>
</div>
```

```
#parent {
  width: 300px;
  height: 300px;
  background-color: red;
}

#child {
  width: 100px;
  height: 100px;
  background-color: aqua;
}
```



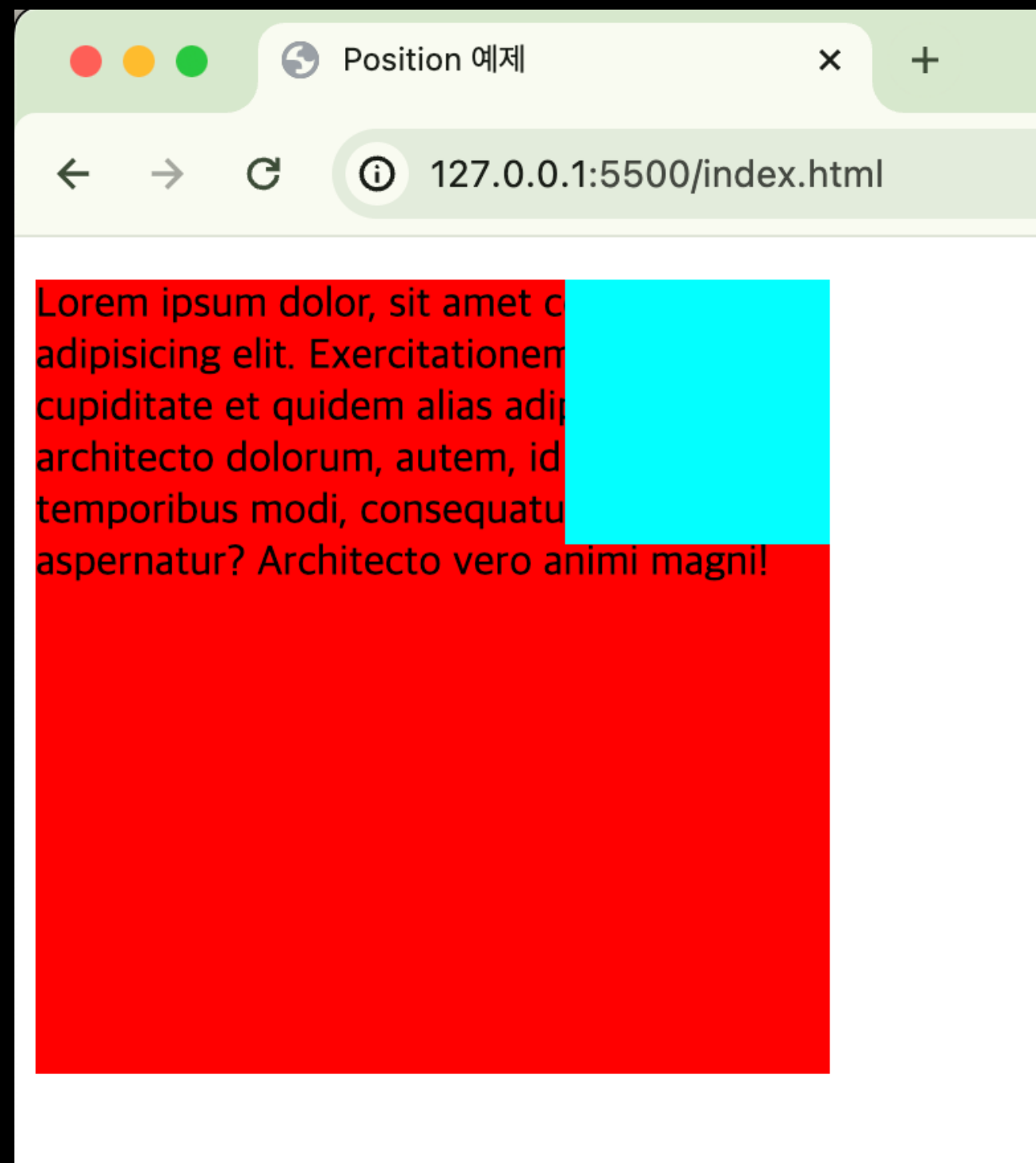
# Position Relative, Absolute



```
#parent {  
  width: 300px;  
  height: 300px;  
  background-color: red;  
  
  position: relative;  
}  
  
#child {  
  width: 100px;  
  height: 100px;  
  background-color: aqua;  
  
  position: absolute;  
}
```



# Position Relative, Absolute

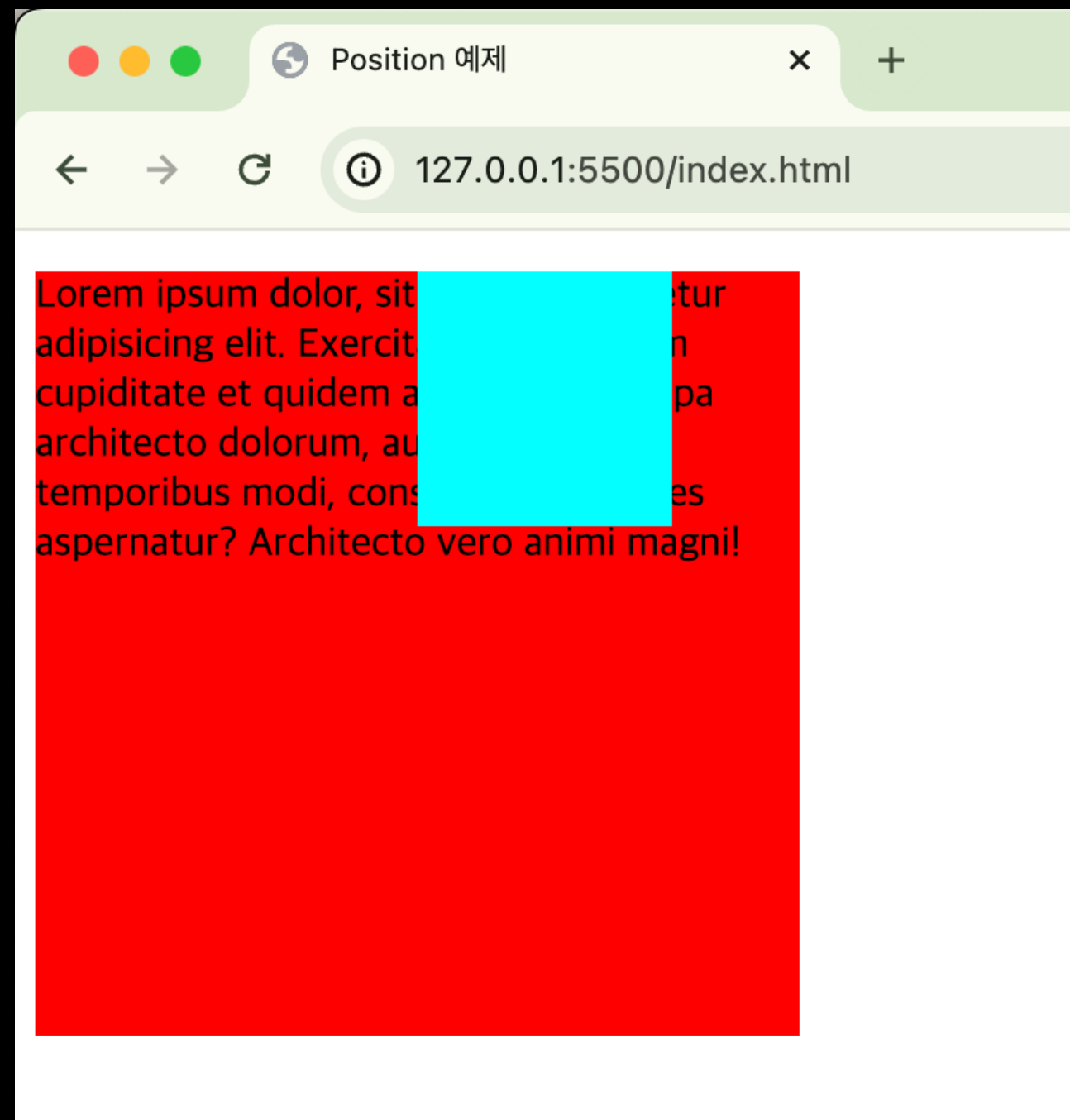


```
#parent {  
  width: 300px;  
  height: 300px;  
  background-color: red;  
  
  position: relative;  
}  
  
#child {  
  width: 100px;  
  height: 100px;  
  background-color: aqua;  
  
  position: absolute;  
  right: 0;  
}
```





# Position Relative, Absolute



```
#parent {  
  width: 300px;  
  height: 300px;  
  background-color: red;  
  
  position: relative;  
}  
  
#child {  
  width: 100px;  
  height: 100px;  
  background-color: aqua;  
  
  position: absolute;  
  right: 50px;  
}
```



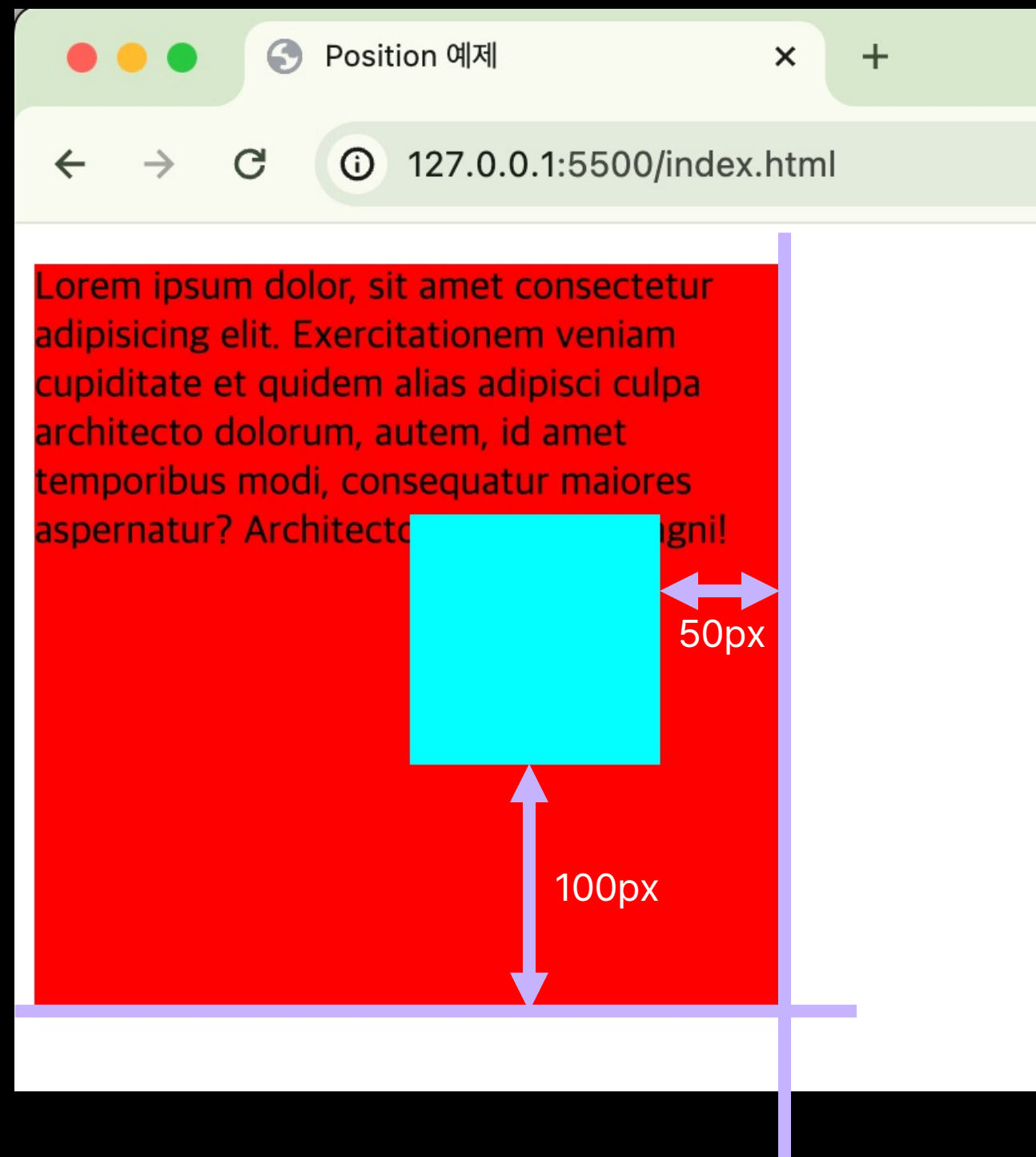
# Position Relative, Absolute



```
#parent {  
  width: 300px;  
  height: 300px;  
  background-color: red;  
  
  position: relative;  
}  
  
#child {  
  width: 100px;  
  height: 100px;  
  background-color: aqua;  
  
  position: absolute;  
  right: 50px;  
}
```



# Position Relative, Absolute



```
#parent {  
  width: 300px;  
  height: 300px;  
  background-color: red;  
  
  position: relative;  
}  
  
#child {  
  width: 100px;  
  height: 100px;  
  background-color: aqua;  
  
  position: absolute;  
  right: 50px;  
  bottom: 100px;  
}
```



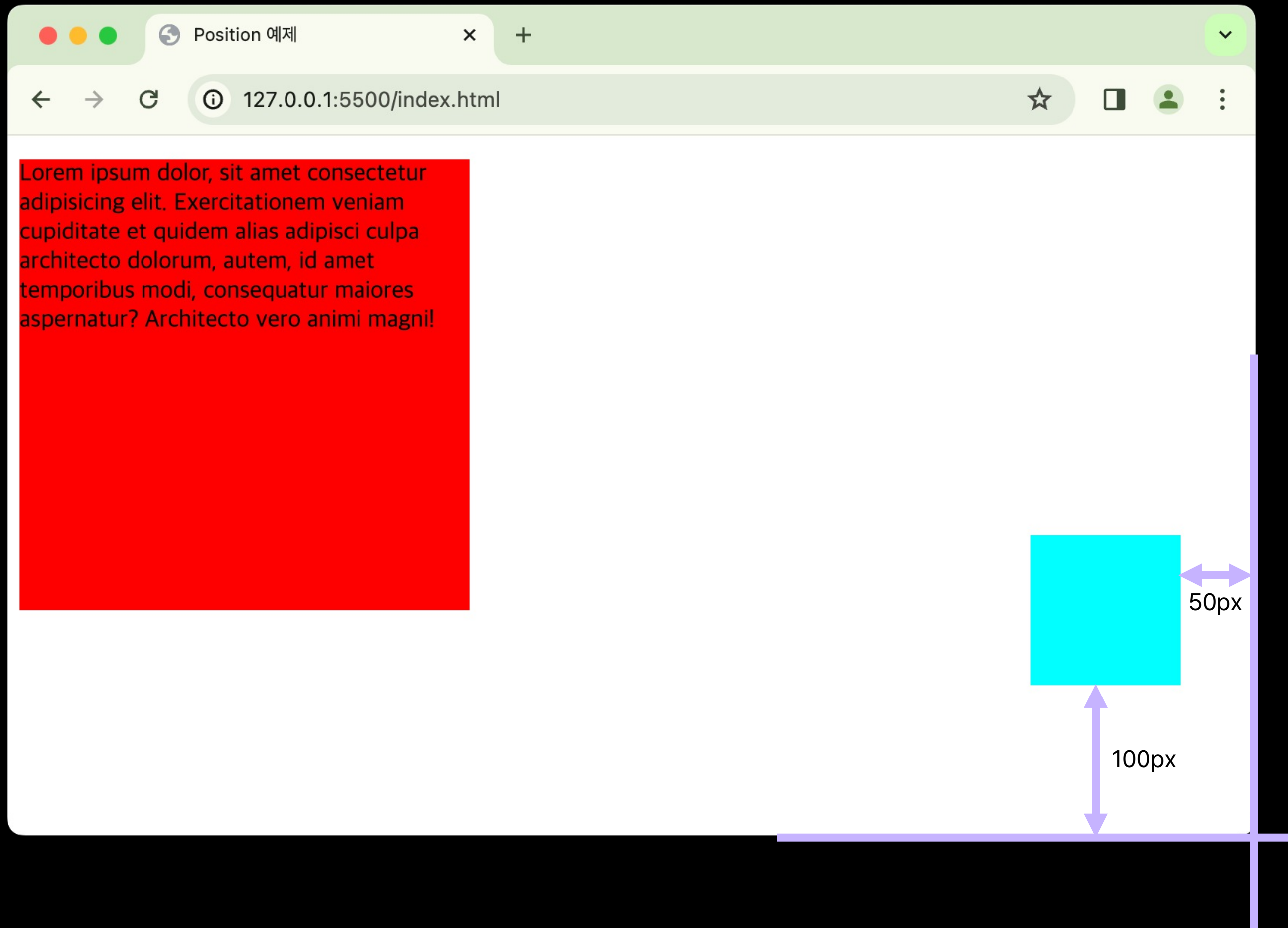


# Position Fixed

HTML/CSS

02 CSS 기초

9. Position



```
#parent {  
  width: 300px;  
  height: 300px;  
  background-color: red;  
  
  position: relative;  
}  
  
#child {  
  width: 100px;  
  height: 100px;  
  background-color: aqua;  
  
  position: fixed;  
  right: 50px;  
  bottom: 100px;  
}
```



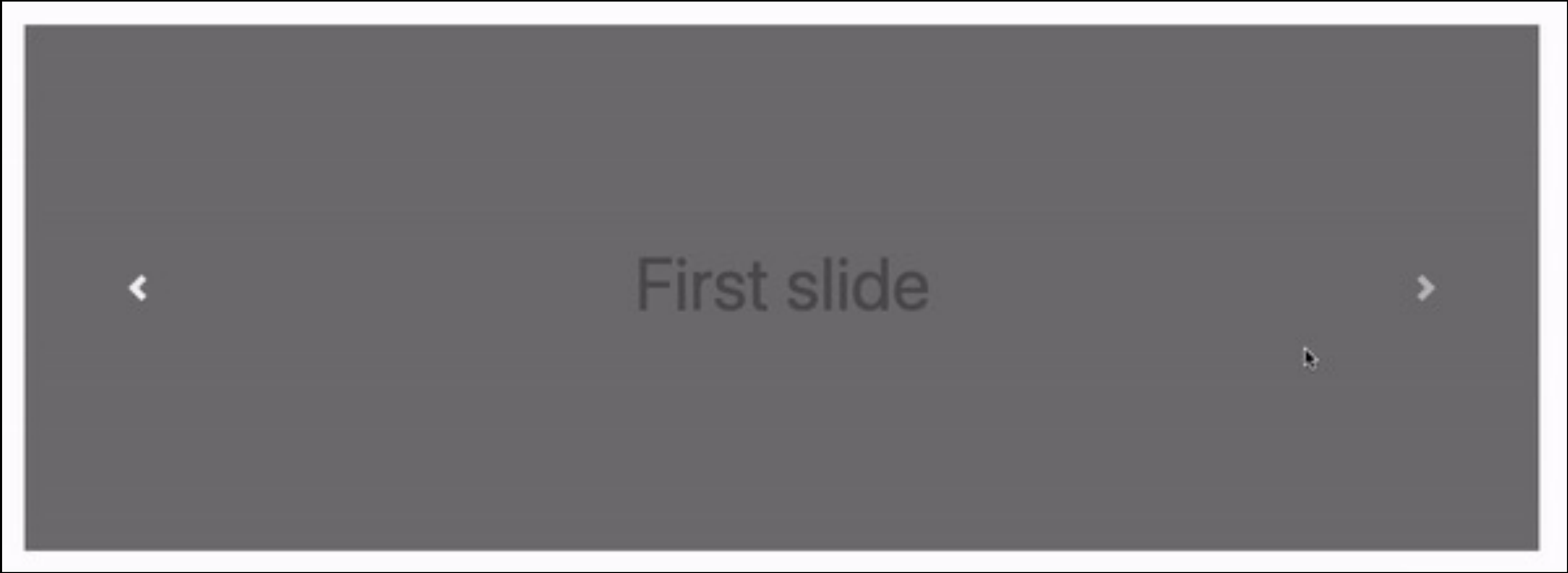


Image Carousel

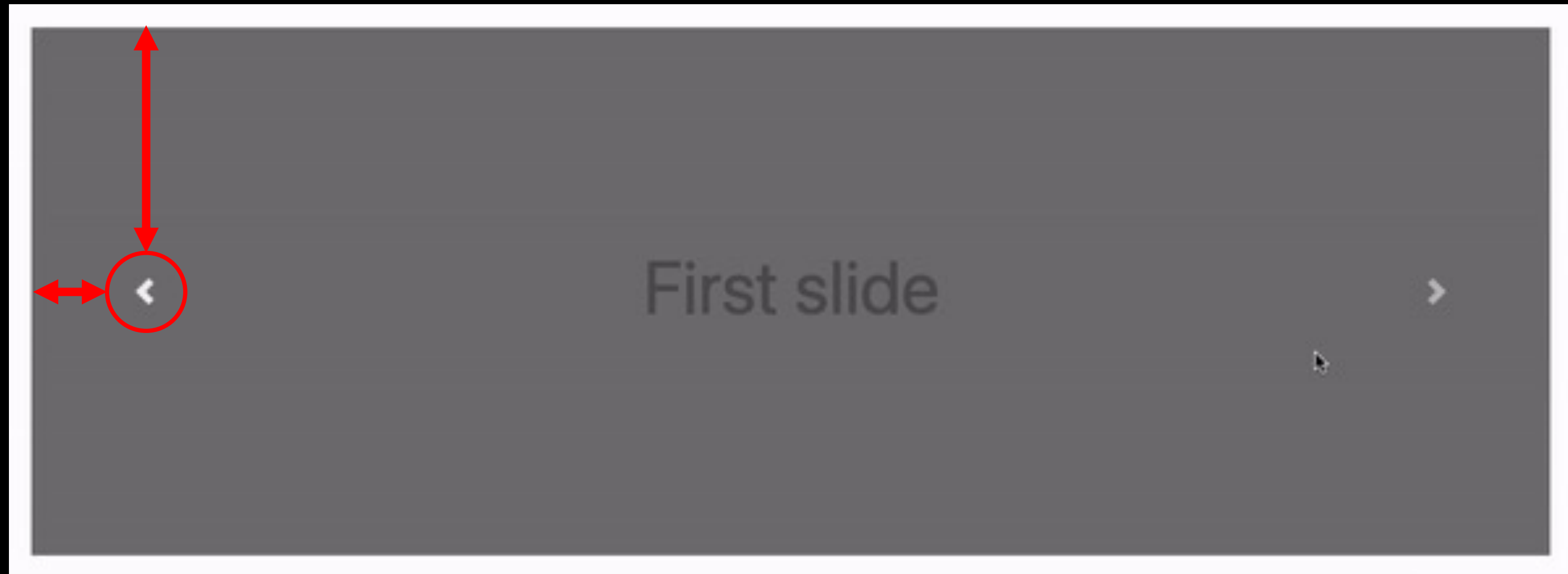


Image Carousel

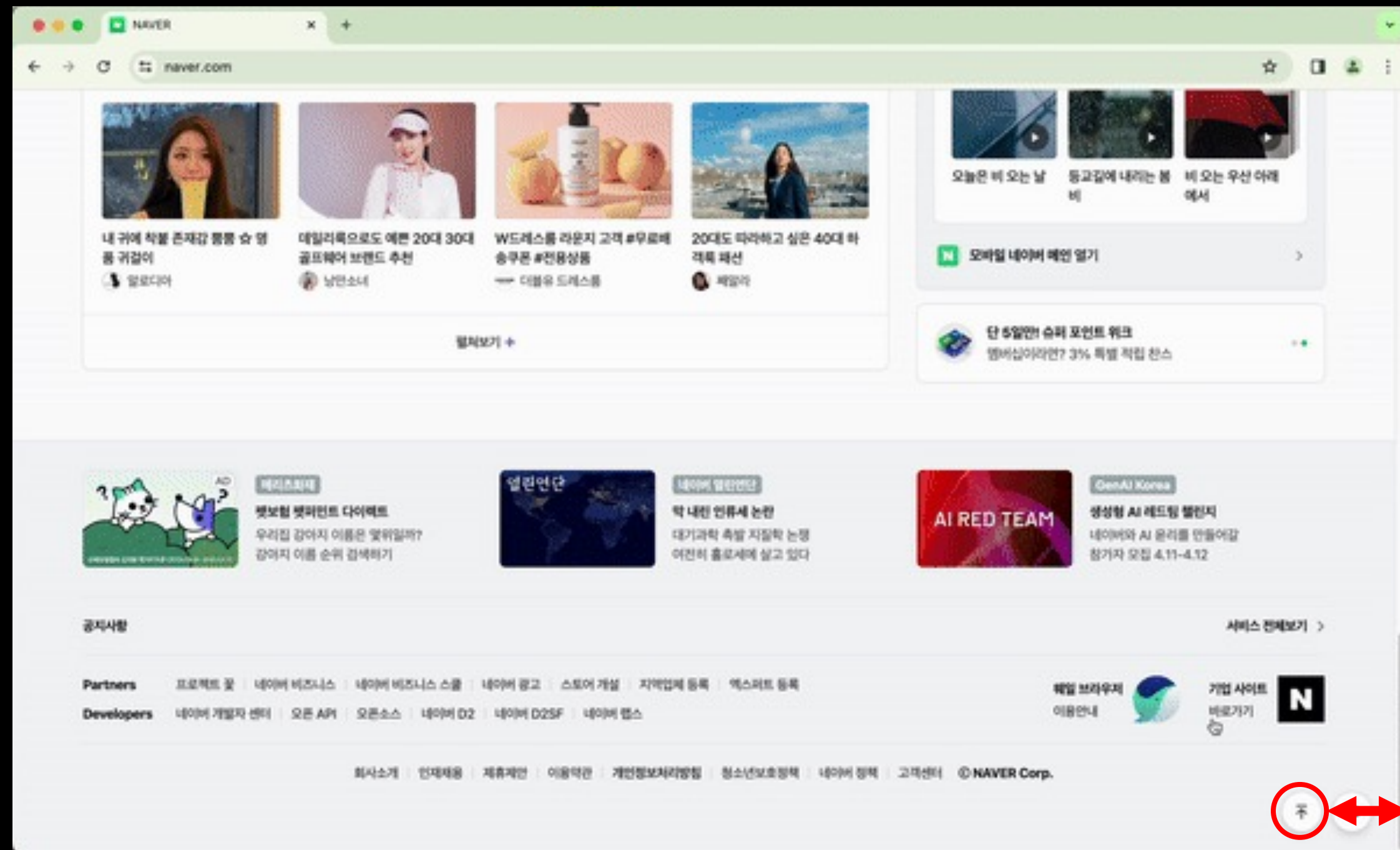


# Position Fixed 사용 예시

HTML/CSS

02 CSS 기초

9. Position



'맨 위로' 버튼